Universidad de Buenos Aires



Facultades de Ciencias Económicas, Ciencias Exactas y Naturales e Ingeniería

Maestría en Seguridad Informática

Tesis de Maestría

"Estrategia de implementación para la gestión segura de dependencias de software: integración de marcos y validación mediante prueba de concepto"

Autor: Lic. Jonathan Leonel Slipak

Director: Mg. Sergio Correa Rovira

Cohorte 2024

Año de presentación: 2026

Declaración jurada de origen de los contenidos

"Por medio de la presente, el autor manifiesta conocer y aceptar el Reglamento de Trabajos Finales vigente y se hace responsable que la totalidad de los contenidos del presente documento son originales y de su creación exclusiva, o bien pertenecen a terceros u otras fuentes, que han sido adecuadamente referenciados y cuya inclusión no infringe la legislación Nacional e Internacional de Propiedad Intelectual."

_____

Slipak Jonathan Leonel
DNI : 28750112

Declaración sobre el uso de herramientas de inteligencia artificial

Para la elaboración gráfica de las Figuras 1, 2 y 3, así como para la revisión editorial, la reorganización de contenidos y el apoyo en la comprobación de referencias y de la *PoC*, se utilizó *ChatGPT/Codex* de *OpenAI*. La herramienta también asistió en la preparación de la arquitectura de referencia. La validación académica de la versión presentada, la interpretación de los resultados y la responsabilidad por el contenido corresponden al autor.

Resumen

La incorporación de paquetes de terceros introduce riesgos de vulnerabilidades conocidas, código malicioso, compromiso de identidades, manipulación de artefactos, abandono y licenciamiento. Este trabajo propone una estrategia de implementación para gestionar esos riesgos en dependencias directas y transitivas distribuidas mediante gestores de paquetes. La metodología combina análisis documental de marcos, estándares y guías con una prueba de concepto local. La propuesta utiliza las funciones de *OWASP SAMM* como referencia, sitúa la gobernanza de forma transversal y relaciona amenazas, controles, responsables, herramientas y evidencias. La validación utiliza una política versionada y diez escenarios de laboratorio, analizados con *npm*, consultas al registro y el escáner *Trivy* 0.76.2. En la corrida registrada el 6 de noviembre de 2026, las diez decisiones coincidieron con las esperadas: siete escenarios riesgosos fueron bloqueados, dos derivados a revisión manual y el escenario limpio permitido. La prueba emplea datos actuales de las fuentes consultadas y conserva sus versiones y evidencias; los resultados pueden variar al actualizarse los paquetes o la base de vulnerabilidades. Los scripts de ciclo de vida se deshabilitan durante las instalaciones. La validación demuestra el funcionamiento del flujo sobre estos casos, no su eficacia en producción. El aporte es una propuesta trazable de adopción, con delimitación explícita entre controles generales y capacidades efectivamente demostradas.

Palabras clave: paquetes de software, dependencias transitivas, cadena de suministro, gestión de riesgos, *SBOM*, políticas como código.

Agradecimiento

A quienes ya no existen en este mundo, pero permanecen vivos en mi memoria y en cada paso que doy: a mi amada abuela Clara y a mi querido padre Oscar, cuyo recuerdo y afecto siguen guiando mi camino; y a quienes me acompañan en el presente, especialmente a mi madre Matilde, cuya disponibilidad, integridad y compromiso han sido pilares esenciales en mi vida y en la persona que hoy soy.

ÍNDICE

Declaración jurada de origen de los contenidos II

Declaración sobre el uso de herramientas de inteligencia artificial II

Resumen III

Agradecimiento IV

Nómina de abreviaturas XI

1. Tema y fundamentos **1**
1.1. Dimensión global del problema 1

2. Planteamiento del problema **4**
2.1. Caso ilustrativo: compromiso de la dependencia *Axios* 5

3. Objetivos **7**
3.1. Objetivo general 7
3.2. Objetivos específicos 7

4. Alcance **8**

5. Marco teórico **9**
5.1. Programación modular y reutilización de software 9
5.2. Dependencias de software y definiciones operativas 9
5.3. Seguridad en la cadena de suministro de software 10
5.3.1. Clases de riesgo y controles diferenciados 10
5.3.2. Amenazas por etapa de la cadena de suministro 10
5.4. Vulnerabilidades conocidas y priorización 12
5.5. *Software Bill of Materials* y artefactos de evidencia 12
5.6. Modelo de ciclo de vida utilizado como referencia 13
5.7. Naturaleza y alcance de las referencias analizadas 14
5.7.1. *NIST SSDF* 14
5.7.2. *OWASP SAMM* 15
5.7.3. *OWASP SCVS* 15
5.7.4. *SLSA* y las garantías sobre la construcción 15
5.7.5. *ISO* y tratamiento organizacional del riesgo 16
5.7.6. Guías *CISA*, *NTIA* y *CIS* 16

5.7.7. Formatos y herramientas de apoyo 16
5.8. Comparación de aportes por función del ciclo de vida 16
5.9. *DevSecOps* y automatización de controles 18
5.10. Gestión del riesgo y monitoreo de dependencias 19
5.11. Fundamentos de la estrategia propuesta 19

6. Enfoque metodológico **20**
6.1. Tipo y enfoque de investigación 20
6.2. Diseño de la investigación 20
6.3. Pregunta de investigación 21
6.4. Unidad de análisis y alcance 21
6.5. Criterios de selección de marcos y estándares 21
6.6. Procedimiento de análisis comparativo 22
6.7. Procedimiento de construcción de la estrategia 22
6.8. Diseño de la prueba de concepto 22
6.9. Criterios e indicadores de evaluación 23
6.10. Limitaciones metodológicas 23

7. Estrategia de implementación para la gestión segura de dependencias **25**
7.1. Principios rectores 25
7.2. Gobernanza, roles y responsabilidades 25
7.3. Ciclo de vida de las dependencias 26
7.4. Arquitectura de referencia e implementación mínima 26
7.5. Inventario consolidado de controles 28
7.6. Preparación y definición de políticas 30
7.7. Selección e incorporación segura de dependencias 31
7.8. Configuración segura del repositorio 31
7.9. Gestión de entornos diferenciados 31
7.10. Validación local y revisión colaborativa 32
7.11. Automatización de controles en *CI/CD* 32
7.12. Generación y gestión del *SBOM* 32
7.13. Protección de la compilación y los artefactos 33
7.14. Firma, procedencia y distribución 33
7.15. Despliegue seguro y monitoreo continuo 33
7.16. Actualización, sustitución y retiro de dependencias 34
7.17. Gestión de incidentes 34
7.18. Capacitación y concientización 34
7.19. Niveles de adopción de la estrategia 35
7.20. Indicadores y mejora continua 35

8. Prueba de concepto **36**
8.1. Objetivo de la prueba de concepto 36
8.2. Descripción del entorno controlado 36
8.3. Arquitectura y tecnologías utilizadas 37
8.4. Línea de base y alcance de la comparación 37
8.5. Escenarios de riesgo evaluados 37
8.6. Controles implementados 38
8.7. Integración de controles en el pipeline 38
8.8. Generación de evidencias 39
8.9. Procedimiento de ejecución 39
8.10. Criterios de aceptación 39

9. Resultados y discusión **41**
9.1. Resultados de la corrida real 41
9.2. Concordancia y distribución de decisiones 41
9.3. Resultados por escenario de riesgo 41
9.4. Resumen de resultados del ensayo 42
9.5. Evaluación de los indicadores 42
9.6. Trazabilidad con los marcos y estándares 43
9.7. Cumplimiento de los objetivos de investigación 43
9.8. Discusión de los resultados 44
9.9. Limitaciones de la prueba de concepto 44

10. Conclusiones **46**
10.1. Cumplimiento del objetivo general 46
10.2. Cumplimiento de los objetivos específicos 46
10.3. Principales hallazgos 47
10.4. Aportes de la estrategia propuesta 47
10.5. Limitaciones del trabajo 47
10.6. Recomendaciones para su implementación 48
10.7. Líneas de trabajo futuro 48

Bibliografía **49**

Anexo A. Prueba de concepto y ejecución registrada **54**
A.1. Detalle de implementación, escenarios, evidencias y limitaciones por control . . 54
A.2. Identificación de la versión y disponibilidad 57
A.2.1. Ejecución registrada 57
A.3. Requisitos y procedimiento de ejecución 58
A.4. Demostración y resultados observados 58

A.5. Inventario de evidencias 58
A.5.1. Inventario verificable de la corrida 60

Índice de figuras

1. Indicadores globales del riesgo asociado a terceros y dependencias. Fuente: elaboración propia a partir de [3, 4, 6]. Figura generada con apoyo de *ChatGPT/Codex* de *OpenAI*. 2

2. Amenazas y controles por etapa de la cadena de suministro de software. Fuente: elaboración propia con base en [2, 23, 29, 30]. Figura generada con apoyo de *ChatGPT/Codex* de *OpenAI*. 11

3. Alcances complementarios de los marcos y guías analizados. Fuente: elaboración propia a partir de [1, 2, 7, 28, 29, 30, 41]. Figura generada con apoyo de *ChatGPT/Codex* de *OpenAI*. 18

4. Arquitectura de referencia y representación del modo simulado separado de la *PoC*. Fuente: elaboración propia con base en [1, 2, 29, 45]. 27

Índice de tablas

1. Clases de riesgo consideradas en la estrategia 11
2. Funciones de referencia y actividades de gestión de dependencias 13
3. Marcos, estándares, guías, artefactos y herramientas 14
4. Aportes de las referencias por función del ciclo de vida 17
5. Indicadores utilizados para evaluar la prueba de concepto 23
6. Responsabilidades principales dentro de la estrategia 26
7. Inventario consolidado de controles de la estrategia 29
8. Niveles propuestos para la adopción de la estrategia 35
9. Escenarios utilizados en la prueba de concepto 38
10. Síntesis de resultados de la corrida real 42
11. Trazabilidad de la prueba de concepto con las referencias analizadas 43

A.1. Detalle de implementación, escenarios, evidencias y limitaciones por control (Anexo A) 54
A.2. Resultados de la ejecución de los escenarios reales de laboratorio 59
A.3. Síntesis de la corrida registrada 59
A.4. Inventario verificable de evidencias de la corrida real 60

Nómina de abreviaturas

Sigla	Significado
API	*Application Programming Interface* (interfaz de programación de aplicaciones)
CI	*Continuous Integration* (integración continua)
CD	*Continuous Delivery*/Deployment (entrega o despliegue continuo)
CI/CD	*Continuous Integration*/*Continuous Delivery* or Deployment
CIS	*Center for Internet Security*
CISA	*Cybersecurity and Infrastructure Security Agency*
CVE	*Common Vulnerabilities and Exposures*
CVSS	*Common Vulnerability Scoring System*
DBIR	*Data Breach Investigations Report*
DevOps	*Development and Operations*
DevSecOps	*Development, Security and Operations*
ENISA	*European Union Agency for Cybersecurity*
EPSS	*Exploit Prediction Scoring System*
FIRST	*Forum of Incident Response and Security Teams*
HTTP	*Hypertext Transfer Protocol*
HTTPS	*Hypertext Transfer Protocol Secure*
IEC	*International Electrotechnical Commission*
IEEE	*Institute of Electrical and Electronics Engineers*
ISO	*International Organization for Standardization*
JSON	*JavaScript Object Notation*
MFA	*Multi-Factor Authentication*
MITRE	*MITRE* Corporation
NIST	*National Institute of Standards and Technology*
npm	*Node Package Manager*
NTIA	*National Telecommunications and Information Administration*
NVD	National Vulnerability Database
OPA	*Open Policy Agent*
OpenSSF	*Open Source Security Foundation*
OWASP	*Open Worldwide Application Security Project*
PoC	*Proof of Concept* (prueba de concepto)
RAT	*Remote Access Trojan*
SAMM	*Software Assurance Maturity Model*

Sigla	Significado
SBOM	*Software Bill of Materials*
SCA	*Software Composition Analysis*
SCVS	*Software Component Verification Standard*
SHA-256	*Secure Hash Algorithm* de 256 bits
SLSA	*Supply-chain Levels for Software Artifacts*
SPDX	*Software Package* Data Exchange
SSDF	*Secure Software Development Framework*
SWID	*Software Identification Tag*
VEX	*Vulnerability Exploitability eXchange*

1. Tema y fundamentos

En la actualidad, el *software* ocupa un rol central en prácticamente todos los sectores de la sociedad, desde servicios financieros, salud, industria, administración pública y comunicaciones, hasta infraestructuras críticas. Este crecimiento ha venido acompañado de una transformación en las prácticas de desarrollo, donde la reutilización de componentes de terceros, bibliotecas *open source*, *frameworks* y paquetes externos se ha convertido en un estándar para reducir tiempos de desarrollo, optimizar recursos y acelerar la entrega de valor. Sin embargo, esta dependencia creciente también ha generado nuevos desafíos de seguridad, ya que la confiabilidad de un sistema ya no depende exclusivamente del código desarrollado internamente, sino también de la calidad, mantenimiento, origen y seguridad de todos los componentes externos que forman parte de la aplicación [1].

En este escenario, la gestión de dependencias se ha consolidado como un aspecto crítico dentro de la seguridad del *software*. Cada dependencia incorporada amplía la superficie de ataque del sistema y puede convertirse en un vector de riesgo si presenta vulnerabilidades conocidas, configuraciones inseguras, falta de mantenimiento o si ha sido comprometida en la cadena de distribución. Casos recientes de incidentes en la cadena de suministro de *software* han demostrado que una vulnerabilidad en un componente externo puede propagarse y afectar múltiples sistemas, organizaciones y usuarios finales, generando impactos operativos, económicos y reputacionales significativos. Por ello, la seguridad de la cadena de suministro de *software* (*Software Supply Chain Security*) se ha convertido en una prioridad estratégica tanto para organizaciones privadas como para organismos públicos [1].

La complejidad actual del ecosistema de dependencias agrava este problema. En muchos proyectos modernos, una aplicación puede incorporar cientos o incluso miles de dependencias directas e indirectas, lo que dificulta mantener visibilidad sobre qué componentes se utilizan, en qué versiones, con qué nivel de criticidad y cuáles presentan riesgos potenciales. Esta situación vuelve insuficientes los enfoques tradicionales de gestión manual o correctiva, que suelen actuar una vez detectado el incidente o la vulnerabilidad. Frente a este panorama, resulta necesario avanzar hacia modelos de gestión proactivos, sistemáticos y basados en controles continuos, capaces de anticipar riesgos y establecer mecanismos claros de gobernanza sobre el uso de componentes externos [2].

1.1 Dimensión global del problema

El *Verizon DBIR* 2026 informa que el 48 % de las brechas de su conjunto de datos involucró a terceros, un aumento relativo del 60 % respecto del conjunto anterior [3]. La categoría incluye distintos tipos de terceros y no debe interpretarse como la proporción de brechas causadas exclusivamente por paquetes de software. Por su parte, el informe *Sonatype* 2024 registró 512.847

paquetes maliciosos en su período de observación, con un aumento interanual del 156 % [4]. Son conjuntos y períodos diferentes, útiles para contextualizar el problema y no para estimar el riesgo de la *PoC*.

El problema combina ataques deliberados con una deuda persistente de mantenimiento. *Sonatype* observó que el 80 % de las dependencias de aplicaciones permanecía sin actualizar por más de un año, aun cuando el 95 % de las versiones vulnerables analizadas contaba con alternativas más seguras [4]. Desde la perspectiva del ataque, *ENISA* determinó, sobre una muestra de 24 incidentes de cadena de suministro, que el 66 % se concentró en el código del proveedor y que el 58 % buscó acceder a datos de clientes, incluida información personal y propiedad intelectual [5].

La materialidad también es económica. El informe global *Cost of a Data Breach 2025* estimó un costo promedio de *USD* 4,44 millones por brecha [6]. Esta cifra no es exclusiva de incidentes de cadena de suministro, pero permite dimensionar el orden de magnitud de una brecha cuando se combina con la creciente participación de terceros observada por *Verizon*. En conjunto, los datos justifican tratar la gestión de dependencias como un proceso de gobierno y riesgo continuo, y no como una tarea operativa aislada.



Figura 1: Indicadores globales del riesgo asociado a terceros y dependencias. Fuente: elaboración propia a partir de [3, 4, 6]. Figura generada con apoyo de *ChatGPT/Codex* de *OpenAI*.

El *SBOM*, definido en la sección 5.5, aporta información sobre la composición del producto y permite relacionar avisos con componentes concretos. Su utilidad depende de la cobertura del inventario, de su asociación con la versión desplegada y de la existencia de procesos que actúen sobre los hallazgos [7].

La integración de controles en solicitudes de cambio y procesos *CI/CD* permite evaluar

una dependencia antes de su promoción. El enfoque *DevSecOps* de la sección 5.9 vincula esa automatización con responsabilidades compartidas y seguimiento de hallazgos [2, 43].

Desde una perspectiva académica, abordar la gestión segura de dependencias como objeto de estudio resulta pertinente por tratarse de un problema actual, transversal y de alto impacto en la seguridad del *software*. Su relevancia no se limita a una tecnología, lenguaje o entorno particular, sino que afecta a organizaciones de distintos sectores que desarrollan, integran o mantienen sistemas basados en componentes externos. Esta investigación permitirá analizar distintos modelos, estándares y buenas prácticas existentes en la industria —como *NIST SSDF*, los lineamientos de *CISA*, *SBOM* y los marcos de verificación de componentes— con el propósito de identificar principios comunes, mecanismos de control y patrones de gestión que puedan articularse dentro de una estrategia integral de implementación [1, 2, 7].

En función de ello, el presente trabajo propone integrar y desarrollar una estrategia de implementación para la gestión segura de dependencias, basada en marcos, estándares y buenas prácticas existentes y orientada a mejorar la gobernanza, la trazabilidad, la automatización y el control de riesgos dentro del ciclo de vida del *software*. La estrategia establecerá lineamientos para la incorporación, evaluación, monitoreo, actualización y retiro de dependencias, y su aplicabilidad será validada mediante una prueba de concepto en un entorno controlado. El aporte de la investigación no radica en crear un marco nuevo, sino en convertir referencias dispersas y complementarias en un enfoque aplicable y verificable para organizaciones que buscan fortalecer la seguridad de su cadena de suministro [8, 9].

2. Planteamiento del problema

En el desarrollo moderno de *software*, la programación modular y el uso de dependencias de terceros se han consolidado como elementos esenciales para construir aplicaciones de forma ágil, escalable y mantenible. Actualmente, la mayoría de los proyectos desarrollados con tecnologías como *PHP*, *Ruby*, *Node.js*, *Ruby on Rails*, *Laravel*, *Django*, *Spring Boot* y *Next.js* dependen en gran medida de bibliotecas, paquetes, *gems*, *plugins* y módulos externos para implementar funcionalidades de autenticación, acceso a bases de datos, validaciones, cifrado, interfaces gráficas, *APIs* y procesos de despliegue. Esta reutilización de componentes permite reducir tiempos de desarrollo, optimizar recursos y aprovechar soluciones maduras ampliamente adoptadas por la comunidad. Sin embargo, también implica trasladar parte de la confianza del sistema hacia terceros, incrementando la complejidad y los riesgos asociados a la seguridad del *software* [1].

En la práctica, los ecosistemas modernos de desarrollo están profundamente apoyados en gestores de dependencias como *Composer* en *PHP*, *Bundler* en *Ruby*, *npm* o *yarn* en *Node.js*, *pip* en *Python* y *Maven* o *Gradle* en *Java*. Estas herramientas facilitan la instalación y actualización automática de componentes externos, pero también generan estructuras de dependencias complejas donde coexisten dependencias directas e indirectas (transitivas). Como consecuencia, una aplicación puede incorporar cientos o miles de componentes externos sin que los equipos de desarrollo tengan visibilidad completa sobre su origen, mantenimiento, historial de vulnerabilidades, compatibilidad o nivel real de exposición al riesgo. Esta falta de trazabilidad constituye uno de los principales desafíos de la seguridad en la cadena de suministro de *software* [7].

El problema se agrava porque cada dependencia de terceros representa un posible punto de entrada para amenazas. Un paquete vulnerable, malicioso, abandonado o comprometido puede afectar directamente la integridad, disponibilidad y confidencialidad de los sistemas que lo utilizan. La historia reciente de la ciberseguridad ha demostrado que ataques a la cadena de suministro de *software* pueden tener efectos masivos y propagarse rápidamente a través de componentes ampliamente reutilizados. En ecosistemas abiertos como *npm*, diversos estudios han evidenciado que un número reducido de paquetes o mantenedores puede impactar una gran parte del ecosistema, generando riesgos sistémicos difíciles de controlar. Además, las dependencias transitivas amplifican la propagación de vulnerabilidades y dificultan aún más la remediación efectiva [10, 11].

En tecnologías como *Ruby on Rails* o *Laravel*, gran parte de la funcionalidad de una aplicación se apoya en *gems*, paquetes y extensiones de terceros. Esto significa que la estabilidad y seguridad del proyecto dependen no solo de las decisiones del equipo de desarrollo, sino también de la calidad del mantenimiento externo, la frecuencia de actualización de librerías y la confianza depositada en mantenedores de código abierto. En muchos casos, los equipos des-

conocen con precisión qué dependencias transitivas están siendo utilizadas, cuáles ya no tienen soporte activo o cuáles presentan vulnerabilidades conocidas. Esta situación genera una falsa sensación de seguridad y dificulta la toma de decisiones preventivas [1].

Otro aspecto central del problema es que, en numerosos entornos de desarrollo, la gestión de dependencias continúa siendo tratada como una tarea operativa y no como un proceso estratégico de seguridad. Frecuentemente, las dependencias se incorporan en función de la necesidad funcional inmediata, sin contar con criterios formales de evaluación de riesgo, validación de origen, control de integridad o seguimiento de actualizaciones críticas. A esto se suma que muchas organizaciones no integran controles automatizados dentro de sus *pipelines* de integración y despliegue continuo (*CI/CD*), lo que impide detectar vulnerabilidades en etapas tempranas del ciclo de desarrollo. Como resultado, componentes inseguros pueden llegar a producción sin haber sido adecuadamente evaluados [1, 2].

La generación de inventarios, el análisis de componentes y la conservación de evidencias deben articularse con reglas de decisión y responsables. Las definiciones de la sección 5.2 permiten distinguir visibilidad de trazabilidad; disponer de una lista de paquetes no acredita por sí solo que su incorporación haya sido evaluada.

Por lo tanto, el problema de investigación no radica en la inexistencia de marcos de referencia, sino en la fragmentación de su cobertura y en la dificultad de trasladarlos a una estrategia de implementación coherente. Aunque existen referencias robustas para el desarrollo seguro, la madurez organizacional, la verificación de componentes, la integridad del *build* y la trazabilidad, ninguna aborda por sí sola todo el ciclo de vida de las dependencias. Esta dispersión limita la visibilidad sobre los componentes utilizados, dificulta la priorización de riesgos y reduce la resiliencia frente a amenazas de la cadena de suministro. En consecuencia, se requiere articular los enfoques existentes en una estrategia que establezca lineamientos claros para la incorporación, monitoreo, actualización y control seguro de dependencias [1, 2, 7, 8].

2.1 Caso ilustrativo: compromiso de la dependencia *Axios*

Un caso reciente que refleja de manera concreta la importancia de la gestión segura de dependencias es el incidente ocurrido con *Axios*, una de las bibliotecas más utilizadas dentro del ecosistema de *Node.js* para realizar solicitudes *HTTP*. *Axios* es una dependencia ampliamente adoptada tanto en aplicaciones *backend* como *frontend*, utilizada en proyectos desarrollados con tecnologías como *React*, *Vue.js*, *Next.js* y diversos servicios de *APIs* y *microservicios*. Su popularidad se debe a que simplifica tareas críticas de comunicación entre sistemas, manejo de respuestas y consumo de servicios externos, por lo que se ha convertido en un componente base dentro de miles de proyectos de *software* [12].

La relevancia de este caso radica en que *Axios* no es una dependencia marginal o de uso limitado, sino un paquete ampliamente distribuido y utilizado por una gran cantidad de desarrolladores, organizaciones y productos en producción. Esto implica que cualquier incidente de seguridad que afecte su cadena de distribución tiene un potencial de impacto masivo. A fines

de marzo de 2026, versiones comprometidas del paquete fueron publicadas en el registro *npm* como consecuencia del secuestro de la cuenta de uno de sus mantenedores. Estas versiones incluyeron una dependencia transitiva maliciosa que ejecutaba un *script* automatizado de instalación (*postinstall*), permitiendo la descarga de *malware* de acceso remoto (*RAT*) en sistemas *Windows*, *Linux* y *macOS* [12].

Este episodio resulta especialmente significativo porque demuestra cómo una dependencia confiable, madura y ampliamente utilizada puede transformarse repentinamente en un vector de ataque sin que los equipos de desarrollo lo adviertan de inmediato. En muchos proyectos, las actualizaciones de paquetes se realizan de forma automática o semiautomática, confiando en la reputación del ecosistema y en la estabilidad histórica del componente. Sin embargo, este incidente evidencia que esa confianza, sin mecanismos de validación adicionales, puede derivar en una exposición crítica para toda la cadena de suministro de *software* [12].

Desde una perspectiva de seguridad, el caso de *Axios* pone de manifiesto la necesidad de adoptar un enfoque preventivo y estructurado para la gestión de dependencias. La alta adopción de bibliotecas de terceros en tecnologías modernas implica que el riesgo asociado a una dependencia vulnerable o comprometida no se limita a un único proyecto, sino que puede escalar rápidamente a nivel organizacional e incluso sectorial. Por ello, resulta indispensable implementar prácticas como inventario de componentes, análisis continuo de vulnerabilidades, trazabilidad, validación de origen y automatización de controles dentro del ciclo de vida del desarrollo [2, 7, 12].

En este sentido, el incidente de *Axios* constituye un ejemplo actual y representativo de por qué la gestión segura de dependencias debe considerarse un eje estratégico dentro de la seguridad del *software*. Su análisis ilustra cómo la falta de controles específicos sobre componentes externos puede traducirse en riesgos reales, reforzando la necesidad de articular una estrategia de implementación que combine inventario, validación de procedencia, monitoreo y actualización segura de dependencias críticas [2, 7, 12].

3. Objetivos

3.1 Objetivo general

Analizar e integrar marcos, estándares y guías para proponer una estrategia de implementación que permita gestionar los riesgos asociados a paquetes de software directos y transitivos durante su ciclo de vida, y evaluar un subconjunto de controles mediante una prueba de concepto local.

3.2 Objetivos específicos

- Analizar los riesgos asociados a la incorporación y gestión de dependencias en el desarrollo de *software*.
- Comparar los aportes de marcos, estándares, guías, formatos y herramientas sobre un ciclo de vida de referencia, distinguiendo sus funciones y límites.
- Identificar principios y controles aplicables a las clases de riesgo consideradas, con trazabilidad a sus referencias de fundamento.
- Proponer la arquitectura general de una estrategia de implementación para la gestión segura de dependencias, contemplando gobernanza, ciclo de vida y niveles de adopción.
- Definir lineamientos para traducir la estrategia en controles técnicos concretos a lo largo del ciclo de vida del software, incluyendo *CI/CD*, *SBOM*, firma y procedencia.
- Diseñar, ejecutar y evaluar una prueba de concepto que permita comprobar la consistencia de un subconjunto de controles mediante escenarios y evidencias reproducibles.

4. Alcance

El objeto principal del trabajo son los paquetes de software externos al código propio de un producto, incorporados mediante gestores de paquetes. Se consideran dependencias directas y transitivas de ejecución, desarrollo y construcción, incluidas bibliotecas y frameworks distribuidos de ese modo. La unidad se identifica por ecosistema, nombre, versión y origen; las definiciones operativas se desarrollan en la sección 5.2.

La estrategia aborda su selección, incorporación, verificación, operación, actualización y retiro, con gobernanza transversal. Considera vulnerabilidades conocidas, paquetes maliciosos, compromiso de identidades o registros, manipulación del build o artefactos, abandono y licenciamiento. Los riesgos no se tratan como equivalentes: cada uno requiere controles y evidencias propios.

Los controles organizacionales se formulan como una arquitectura de referencia e inventario de capacidades. Las herramientas se presentan como opciones de implementación y no como productos comparados exhaustivamente. El modelo de ciclo de vida de la sección 5.6 se utiliza de manera consistente para comparar las fuentes y organizar los controles.

La validación empírica se limita a una *PoC JavaScript* ejecutada con *Node.js*, que crea proyectos de laboratorio, instala dependencias seleccionadas con los scripts deshabilitados y analiza los casos con herramientas reales. La mayor parte de los escenarios corresponde a *npm*; S1 utiliza un archivo de proyecto *Maven* con *Log4j* 2.14.1, analizado por *Trivy*. La prueba consulta el registro de *npm* y la base de vulnerabilidades del escáner, por lo que los hallazgos dependen de las fuentes y versiones registradas en cada corrida. No constituye una integración productiva ni amplía las conclusiones a otros gestores.

Quedan fuera del alcance las *APIs* y servicios remotos, plataformas cloud, motores de bases de datos como servicios, firmware, hardware, paquetes del sistema operativo y la seguridad integral de contenedores o infraestructura. Una biblioteca cliente distribuida como paquete sí pertenece al objeto de estudio; el servicio al que se conecta queda excluido. Tampoco se evalúan una implantación productiva, la detección general de malware, ataques reales a registros, conformidad normativa completa, costos o desempeño a escala.

5. Marco teórico

El marco teórico delimita el objeto de estudio, distingue las clases de riesgo y establece una referencia común para comparar las fuentes. Las definiciones de dependencia, *SBOM*, *DevSecOps*, visibilidad y trazabilidad se concentran en este capítulo; los capítulos posteriores aplican estos conceptos a la estrategia y a la prueba de concepto.

5.1 Programación modular y reutilización de software

La programación modular divide un sistema en componentes con responsabilidades definidas y bajo acoplamiento. Esta organización facilita el mantenimiento y permite reutilizar capacidades que otros equipos o comunidades ya desarrollaron [13, 14]. Reutilizar una biblioteca reduce el código propio que debe mantenerse, pero incorpora decisiones y cambios de terceros al producto.

Los gestores de paquetes automatizan la declaración, resolución y obtención de componentes. Para esta investigación, la unidad relevante es el paquete identificable por nombre, versión y ecosistema, junto con sus relaciones de dependencia. Una misma biblioteca puede distribuirse en varios paquetes, y un paquete puede incluir código destinado a la ejecución, al desarrollo o a la construcción del producto.

La reutilización también aparece en servicios remotos, *APIs* y plataformas administradas [17, 18]. Estas relaciones pertenecen al contexto arquitectónico general, pero tienen mecanismos de contratación, disponibilidad y control distintos de los paquetes incorporados al software. Se excluyen de la validación de este trabajo, conforme al capítulo 4.

5.2 Dependencias de software y definiciones operativas

Se entiende por dependencia de software, a los efectos del trabajo, un paquete externo al código propio del producto, distribuido mediante un gestor de paquetes, necesario para construir, probar o ejecutar ese producto. Se consideran componentes de terceros y componentes internos reutilizables cuando se distribuyen por ese mecanismo. La identificación comprende, como mínimo, ecosistema, nombre, versión y origen.

Una dependencia directa es declarada por el proyecto. Una dependencia transitiva es requerida por otra dependencia. El conjunto resuelto forma un grafo que puede diferir de la lista inicial del manifiesto; por ello, la evaluación debe considerar la resolución efectiva y no únicamente las declaraciones directas [10, 11]. En *npm*, el archivo de bloqueo registra versiones resueltas y datos de integridad, aunque su mera presencia no garantiza que esas versiones sean seguras.

Se distingue entre paquetes utilizados en tiempo de ejecución y paquetes de desarrollo o construcción. Estos últimos también pueden afectar el producto, por ejemplo, al ejecutar scripts

durante la instalación o modificar una compilación. Un entorno sin ejecución del paquete en producción puede seguir expuesto durante el build.

Visibilidad significa conocer qué componentes y versiones intervienen en un producto y dónde se utilizan. Trazabilidad significa poder relacionar una decisión, una revisión de código, las entradas de construcción, el artefacto y las evidencias que justifican su aprobación. Un inventario aporta visibilidad; para obtener trazabilidad debe vincularse con identidades, versiones y registros de ejecución. En este trabajo, monitoreo continuo designa una reevaluación periódica o activada por cambios y alertas, con responsables y plazos definidos; no supone una consulta permanente sin interrupciones.

5.3 Seguridad en la cadena de suministro de software

La cadena de suministro de software comprende los actores, componentes, herramientas y procesos que intervienen en la creación, distribución y mantenimiento de un producto. Su seguridad exige controlar las transiciones entre repositorios, registros de paquetes, plataformas de construcción, almacenes de artefactos y entornos de ejecución [23, 26]. Esta investigación analiza esas transiciones en cuanto afectan la incorporación de paquetes.

5.3.1 Clases de riesgo y controles diferenciados

Las siguientes clases separan problemas que pueden coexistir, pero requieren evidencias y tratamientos diferentes. Los códigos R1 a R6 se utilizan en el inventario de controles del capítulo 7. La Tabla 1 constituye una clasificación analítica del trabajo basada en [1, 2, 23, 29]; no es una taxonomía oficial de esos documentos.

Un escáner de vulnerabilidades conocidas no detecta necesariamente un paquete malicioso nuevo. Autorizar un dominio de registro tampoco acredita la identidad del publicador ni impide el compromiso de una cuenta legítima. Del mismo modo, un hash permite comparar contenidos, pero no acredita su origen si el atacante puede sustituir tanto el archivo como la referencia de confianza. Estas diferencias determinan por qué la estrategia combina controles.

5.3.2 Amenazas por etapa de la cadena de suministro

La Figura 2 representa puntos técnicos de exposición. Sus seis bloques no constituyen el modelo de ciclo de vida utilizado para comparar referencias: se asignan a las funciones definidas en la sección 5.6. Un repositorio, por ejemplo, puede intervenir en implementación y verificación, mientras sus permisos se gobiernan de manera transversal.

En código fuente y registros se consideran cambios no revisados, suplantación de paquetes, compromiso de cuentas y resolución de versiones no aprobadas. La revisión por pares, el control de identidades, las fuentes autorizadas y los archivos de bloqueo cubren aspectos diferentes de esa exposición [23, 30]. Durante la construcción, se añaden riesgos de ejecución de scripts, uso de secretos y alteración de runners o herramientas.

En distribución y operación se debe comprobar que el artefacto recibido sea el aprobado

Tabla 1: Clases de riesgo consideradas en la estrategia

Clase	Condición y consecuencia	Tratamiento principal
R1 Vulnerabilidad conocida	Defecto divulgado en una versión; su explotación depende del uso y de la exposición.	*SCA*, análisis de afectación, actualización o mitigación.
R2 Paquete malicioso	Código introducido con intención dañina, incluido un script de instalación. Puede no tener *CVE*.	Control de origen y nombres, revisión de contenido y scripts, aislamiento y análisis especializado.
R3 Compromiso de cuenta o registro	Uso no autorizado de identidades de mantenedores, tokens o canales de publicación.	*MFA*, mínimo privilegio, publicación confiable, auditoría y respuesta sobre credenciales.
R4 Manipulación del build o artefacto	Alteración de entradas, pipeline, entorno de construcción o producto distribuido.	Protección del build, hashes con referencia confiable, firma y verificación de procedencia.
R5 Abandono o falta de soporte	Ausencia de mantenimiento o soporte que dificulta corregir defectos futuros.	Evaluación de mantenimiento, propietario, plan de sustitución y revisión contextual.
R6 Licenciamiento	Condiciones de uso o distribución incompatibles con la política de la organización.	Identificación de licencias y revisión por el responsable de cumplimiento. No se mide con *CVSS*.



Figura 2: Amenazas y controles por etapa de la cadena de suministro de software. Fuente: elaboración propia con base en [2, 23, 29, 30]. Figura generada con apoyo de *ChatGPT/Codex* de *OpenAI*.

y que las nuevas alertas puedan relacionarse con las aplicaciones desplegadas. La verificación previa a la promoción, la conservación de evidencias y la capacidad de actualizar o retirar componentes completan el tratamiento. El detalle organizacional se desarrolla en el capítulo 7.

5.4 Vulnerabilidades conocidas y priorización

Una vulnerabilidad es una debilidad que puede explotarse para afectar la confidencialidad, integridad o disponibilidad. Un identificador *CVE* permite referenciar un registro; no constituye una medida de severidad ni demuestra que una aplicación concreta sea explotable [33]. *Log4Shell*, registrado como *CVE*-2021-44228, afecta a *Apache Log4j* y se utiliza aquí como ejemplo documental [34].

CVSS expresa severidad técnica mediante un sistema de puntuación. Deben registrarse su versión, vector y fuente para interpretar el valor. La política de una organización puede utilizarlo como entrada, junto con exposición, criticidad, explotación conocida y mitigaciones. El valor *CVSS* no equivale por sí solo a riesgo de negocio [48, 49]. En el ensayo real, *Trivy* aporta severidad y puntuaciones de sus avisos; la *PoC* bloquea los hallazgos críticos o con puntuación igual o superior a 9, y deriva los hallazgos altos o con puntuación igual o superior a 7 a revisión manual. No calcula vectores ni sustituye el análisis contextual.

Las dependencias transitivas definidas en la sección 5.2 amplían el conjunto que debe analizarse. La correlación requiere identificar correctamente paquete, versión y ecosistema, y después determinar si el uso efectivo del componente alcanza la funcionalidad afectada. Un inventario incompleto o una asociación incorrecta entre avisos y paquetes puede producir omisiones o falsos positivos.

La priorización incorpora severidad y disponibilidad de corrección. Una vulnerabilidad confirmada como explotada puede recibir tratamiento urgente aunque no supere un umbral numérico; una alerta sin afectación demostrada requiere análisis. La remediación debe conservar pruebas funcionales y registrar la decisión, el responsable y el plazo [2]. La *PoC* consume reportes de *Trivy* y conserva sus referencias, pero no mide la calidad ni la cobertura general de sus fuentes.

5.5 *Software Bill of Materials* y artefactos de evidencia

Un *Software Bill of Materials* (*SBOM*) es un inventario estructurado y legible por máquina de los componentes de un producto y de sus relaciones. Su alcance debe declararse: lo observado en código fuente, en construcción o en un artefacto distribuido puede diferir. Los elementos mínimos de *NTIA* incluyen identificación de componentes, versiones y relaciones, además de datos sobre la autoría y generación del inventario [36]. Un *SBOM* no es un marco de gestión ni una herramienta de análisis.

CycloneDX y *SPDX* son especificaciones para representar e intercambiar esta información [38, 39]. El archivo generado es el artefacto; el generador de *SBOM* es la herramienta; *SCVS*

proporciona requisitos de verificación. Separar esas categorías evita equiparar la existencia de un *JSON* con el cumplimiento de una práctica de seguridad.

Para uso operativo, el inventario debe identificar componentes directos y transitivos, asociarse con la versión y el hash del producto, y conservar la información necesaria para evaluar licencias y vulnerabilidades. Su calidad requiere validar el esquema y contrastar la cobertura con los componentes realmente resueltos. La firma y su verificación agregan autenticidad a la evidencia, pero no prueban que el inventario sea completo [1, 8].

La procedencia describe cómo y con qué entradas se obtuvo un artefacto. Es diferente del inventario de componentes. Una atestación es una declaración sobre un sujeto identificable; cuando está firmada y se verifica contra una identidad de confianza, permite comprobar quién emitió esa declaración. El *SBOM*, la procedencia, el informe de política y los hashes cumplen funciones complementarias. El modo real de la *PoC* conserva reportes del escáner, resultados y un manifiesto de hashes, pero no genera un *SBOM* ni procedencia. El modo simulado separado genera representaciones demostrativas de estos artefactos; no se utilizan como resultados del ensayo real.

5.6 Modelo de ciclo de vida utilizado como referencia

Se adoptan las funciones de *OWASP SAMM* 2 como esquema de referencia: Diseño, Implementación, Verificación y Operación, con Gobernanza transversal [28]. *SAMM* es un modelo de madurez, no una secuencia rígida de fases. La organización temporal que sigue es una adaptación analítica para este trabajo; las verificaciones pueden repetirse durante la implementación y la operación puede originar nuevos requisitos. La Tabla 2 sintetiza la aplicación de estas funciones a la gestión de dependencias.

Tabla 2: Funciones de referencia y actividades de gestión de dependencias

Función	**Aplicación en el trabajo**	**Salida o decisión**
D Diseño	Definir necesidad, criticidad, alternativas y requisitos de incorporación.	Criterios de aceptación y solicitud de dependencia.
I Implementación	Declarar y resolver paquetes, proteger repositorios y construir artefactos.	Versiones fijadas, artefacto e inventario.
V Verificación	Evaluar componentes, políticas, integridad y evidencia antes de promover.	Permitir, bloquear o requerir revisión documentada.
O Operación	Relacionar versiones desplegadas con alertas; actualizar, responder y retirar.	Inventario operativo, remediación y aprendizaje.
G Gobernanza transversal	Asignar responsables, políticas, excepciones, capacitación e indicadores.	Reglas y decisiones auditables para todas las funciones.

La selección comienza en Diseño; su incorporación y construcción se realizan en Implementación; Verificación aplica criterios de aprobación; Operación vuelve a evaluar el componente hasta su retiro. Una actualización reinicia el recorrido cuando cambia el riesgo o la composición. Gobernanza determina quién puede decidir y bajo qué condiciones durante todo el ciclo.

Los grupos PO, PS, PW y RV del *SSDF* se utilizan como fundamento complementario y no se reinterpretan como fases consecutivas. Preparar la organización y proteger el software son actividades transversales; producir software seguro y responder a vulnerabilidades aportan prácticas a varias funciones [2]. Esta distinción permite comparar referencias diferentes sobre un mismo esquema.

5.7 Naturaleza y alcance de las referencias analizadas

La Tabla 3 distingue propósito, resultado esperado y límite de uso para cada referencia. Las correspondencias posteriores expresan la selección del trabajo; no equivalencias normativas ni declaraciones de certificación.

Tabla 3: Marcos, estándares, guías, artefactos y herramientas

Categoría	**Referencias**	**Función en la estrategia**
Marco de prácticas	*NIST SSDF* 1.1	Organizar prácticas de desarrollo seguro y responsabilidades.
Modelo de madurez	*OWASP SAMM* 2	Evaluar capacidades y planificar mejoras; referencia funcional del ciclo.
Estándar de verificación	*OWASP SCVS* 1.0	Definir requisitos verificables sobre componentes y cadena de suministro.
Especificación de garantías	*SLSA* 1.0, Build Track	Establecer garantías progresivas sobre construcción y procedencia.
Normas y guías de gestión	*ISO/IEC* 27034-1 e *ISO* 31000	Integrar seguridad de aplicaciones y tratamiento del riesgo.
Guías técnicas	*CISA*/*NSA*/*ODNI*, *CIS* y *NTIA*	Orientar implementación y requisitos de información.
Artefactos y formatos	*SBOM*; *CycloneDX* 1.6 y *SPDX* 3.0	El *SBOM* es evidencia; las especificaciones definen su representación.
Herramientas	*OpenSSF Scorecard*, *SCA*, *OPA* y generadores de *SBOM*	Recolectar señales, analizar componentes y ejecutar reglas.

5.7.1 *NIST SSDF*

SSDF 1.1 reúne prácticas en Prepare the Organization (PO), Protect the Software (PS), Produce Well-Secured Software (PW) y Respond to Vulnerabilities (RV). PO aporta requisitos, roles, herramientas y criterios; PS protege código y versiones; PW incorpora desarrollo y componentes seguros; RV aborda identificación, tratamiento y causas de vulnerabilidades [2]. Su alcance es el desarrollo seguro en general.

Para dependencias se seleccionan PW.4 como fundamento de la adquisición y mantenimiento de componentes, PS.2 para la integridad de versiones y RV.1/RV.2 para identificación y respuesta. La estrategia traduce esas prácticas en decisiones y evidencias concretas. *SSDF* no prescribe un escáner, un umbral *CVSS* único ni una implementación de *npm*; esos parámetros

deben definirse según el contexto.

5.7.2 *OWASP SAMM*

SAMM organiza quince prácticas en cinco funciones y permite evaluar y mejorar capacidades [28]. En esta tesis aporta dos usos: ordenar las actividades mediante el modelo de la sección 5.6 y orientar una adopción gradual. La gobernanza considera estrategia, métricas, políticas y formación, mientras las demás funciones conectan requisitos, construcción, verificación y operación.

La estrategia utiliza esta estructura para asignar responsables y establecer hitos de adopción. Los tres niveles propuestos en el capítulo 7 son una secuencia propia de implementación: no equivalen a una evaluación formal de madurez *SAMM*. La *PoC* local no aporta evidencia suficiente para calificar la madurez de una organización.

5.7.3 *OWASP SCVS*

SCVS 1.0 define requisitos en seis dominios: inventario, *SBOM*, entorno de construcción, gestión de paquetes, análisis de componentes y origen/procedencia. Sus niveles permiten incorporar requisitos de manera progresiva [1]. Es la referencia más específica para convertir la gestión de componentes en verificaciones observables.

Se seleccionan, entre otros, los requisitos 1.1 y 1.7 para conocer versiones e identificadores; 2.1, 2.2 y 2.9 para el inventario estructurado, automatizado y completo; 4.13 y 4.18 para integridad y ejecución de código; 5.4, 5.7, 5.8 y 5.12 para vulnerabilidades, actualización, soporte y licencias; y 6.1 para origen verificable. Una lista de registros permitidos cubre solo parte del problema de origen. La trazabilidad a *SCVS* expresa fundamento, no cumplimiento integral de esos requisitos.

5.7.4 *SLSA* y las garantías sobre la construcción

Se utiliza *SLSA* 1.0, Build Track, como versión documental fija [29]. Build L1 exige que exista procedencia; L2 incorpora procedencia firmada generada por una plataforma alojada; L3 agrega garantías de endurecimiento de la plataforma. La compilación reproducible puede ser útil, pero no debe presentarse como requisito universal de todos esos niveles.

SLSA permite analizar si el artefacto fue construido mediante el proceso esperado. No sustituye el análisis de vulnerabilidades, licencias o mantenimiento. La estrategia propone generar, conservar y verificar procedencia como capacidad organizacional. La declaración *JSON* de la *PoC* no está firmada, no corresponde a una construcción productiva y no acredita un nivel *SLSA*. Se conserva la referencia 1.0 para que la comparación sea reproducible, sin presentarla como la versión más reciente.

5.7.5 *ISO* y tratamiento organizacional del riesgo

ISO/IEC 27034-1 aporta conceptos y procesos para integrar seguridad en la gestión de aplicaciones, incluidas aplicaciones adquiridas o desarrolladas por terceros [42]. Se utiliza como referencia de gobierno y vinculación entre controles y contexto organizacional. El análisis se limita a su alcance conceptual; no constituye una auditoría de todos sus requisitos.

ISO 31000 aporta principios y directrices para establecer contexto, evaluar, tratar y revisar riesgos [47]. Su contribución es justificar prioridades, responsables y aceptación del riesgo residual. No define cómo resolver paquetes ni prescribe políticas de licencias específicas. Estas decisiones se documentan en la estrategia y deben ajustarse a la organización que la adopte.

5.7.6 Guías *CISA*, *NTIA* y *CIS*

Las guías *CISA*/*NSA*/*ODNI* vinculan desarrollo, adquisición, gestión de componentes y respuesta en la cadena de suministro [8, 23]. *NTIA* precisa elementos mínimos de información para el *SBOM* [36]. *CIS* aporta orientación técnica para proteger los entornos y procesos de construcción y despliegue automatizados [30]. Su aporte se utiliza para concretar evidencias y prácticas; una guía de implementación no es equivalente a una herramienta ni a una evaluación de madurez.

5.7.7 Formatos y herramientas de apoyo

CycloneDX 1.6 y *SPDX* 3.0 permiten representar información de componentes [38, 39]. Un generador debe producir un documento válido y suficientemente completo para el caso de uso. Declarar una versión de formato dentro de un archivo no prueba su conformidad: se necesita validación de esquema y revisión de contenido.

OpenSSF Scorecard automatiza comprobaciones sobre prácticas observables de proyectos abiertos, como protección de ramas, permisos de tokens, mantenimiento y fijación de dependencias [41]. Sus resultados son señales para decidir; no certifican la ausencia de código malicioso. *SCA* designa una clase de herramientas para analizar composición y avisos; *OPA* es un motor de políticas que evalúa datos estructurados [44]. El modo real de la *PoC* utiliza *Trivy* para analizar vulnerabilidades y licencias, junto con reglas propias en *JavaScript*; no ejecuta *Scorecard* ni *OPA*.

5.8 Comparación de aportes por función del ciclo de vida

La comparación se realiza sobre las funciones ya definidas en la sección 5.6. La Figura 3 conserva una síntesis visual de focos; debe leerse con la clasificación de la sección 5.7. La Tabla 4 detalla el aporte utilizable de cada referencia y señala expresamente dónde se necesitan capacidades complementarias.

Las celdas describen una correspondencia analítica, no un puntaje ni una afirmación de cobertura completa. Las prácticas de *SSDF* y *SAMM* estructuran responsabilidades; *SCVS* es-

Tabla 4: Aportes de las referencias por función del ciclo de vida

Fuente	Diseño	Implementación	Verificación	Operación	Gobernanza transversal
SSDF [2]	Requisitos para adquirir componentes.	Protección de código y uso de componentes seguros.	Criterios y comprobación de integridad.	Identificación y tratamiento de vulnerabilidades.	Roles, herramientas y criterios organizacionales.
SAMM [28]	Amenazas, requisitos y arquitectura.	Construcción y despliegue seguros.	Evaluación de arquitectura y pruebas.	Incidentes y gestión operativa.	Estrategia, políticas, métricas y formación.
SCVS [1]	Requisitos para aceptar componentes.	Inventario, resolución y entorno de build.	Integridad, análisis y origen verificable.	Actualización de inventarios y soporte.	Política de componentes y aceptación del riesgo.
SLSA 1.0 [29]	Definir garantías esperadas del build.	Plataforma y generación de procedencia.	Comparar procedencia con expectativas.	Verificar antes de consumir artefactos.	Asignar confianza al constructor; no gobierna todo el SDLC.
ISO/IEC 27034 [42]	Contexto de seguridad de la aplicación.	Integración de controles en procesos.	Evaluación según contexto y controles.	Gestión de seguridad durante su vida útil.	Marco organizacional de seguridad de aplicaciones.
CISA/NTIA [8, 23, 36]	Requisitos de información de proveedores.	Inventario y prácticas de suministro.	Revisión de información y evidencia recibida.	Uso del inventario para respuesta.	Coordinación entre productores y consumidores.
CIS [30]	Identificar superficies del pipeline.	Protección de herramientas y entornos.	Comprobaciones técnicas de configuración.	Mantener protegida la plataforma.	Responsabilidades sobre automatización.
Scorecard [41]	Señales para seleccionar proyectos.	Comprobaciones de prácticas del repositorio.	Informe de hallazgos automatizados.	Reevaluación del proyecto observado.	Insumo para decisiones; no modelo de gobierno.
SBOM y formatos [36, 38, 39]	Definir cobertura e identificadores.	Generar inventario del producto.	Validar esquema, cobertura y asociación.	Correlacionar componentes y alertas.	Definir responsables y conservación; el formato no los impone.

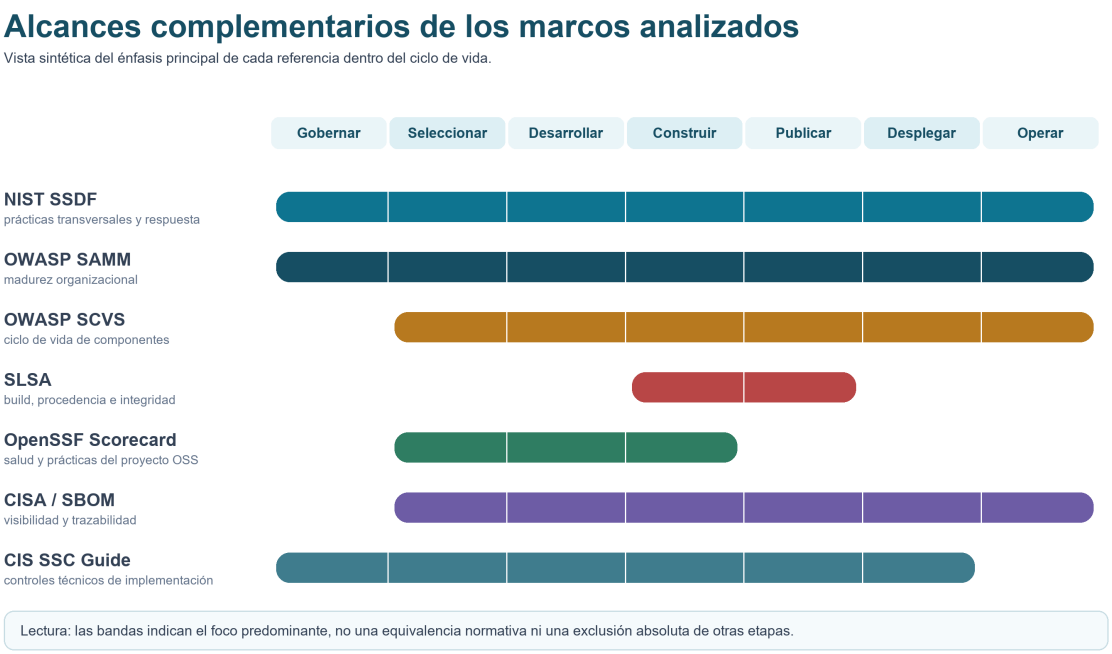


Figura 3: Alcances complementarios de los marcos y guías analizados. Fuente: elaboración propia a partir de [1, 2, 7, 28, 29, 30, 41]. Figura generada con apoyo de *ChatGPT/Codex* de *OpenAI*.

pecifica verificaciones sobre componentes; *SLSA* profundiza la construcción. Las guías, formatos y herramientas materializan parte de esas prácticas. La combinación se justifica por esas funciones diferentes y por los límites identificados.

5.9 *DevSecOps* y automatización de controles

DevSecOps integra la seguridad como responsabilidad compartida entre desarrollo, seguridad y operaciones durante el ciclo de vida [43, 46]. En este trabajo es el enfoque operativo para introducir controles en solicitudes de cambio y pipelines, y para devolver los hallazgos a sus responsables. Automatizar una regla no sustituye la definición de quién acepta una excepción.

Una política como código expresa criterios de aceptación en reglas versionadas y verificables. El pipeline aporta entradas; un motor devuelve permitir, bloquear o revisión manual; la plataforma hace cumplir el resultado. *OPA* es una opción para evaluar esas reglas, mientras la *PoC* combina reglas propias en *JavaScript* con reportes de *Trivy* [44]. Un resultado de revisión manual debe impedir la promoción hasta obtener aprobación registrada.

El fallo del analizador, la falta de datos y la ausencia de hallazgos son estados distintos. En una implementación organizacional, el control debe identificar cada estado y aplicar una política explícita, especialmente si falta el *SBOM*, no responde la fuente de vulnerabilidades o no puede verificarse una firma. La *PoC* no valida estas fallas de integración y las conserva como trabajo posterior.

5.10 Gestión del riesgo y monitoreo de dependencias

El tratamiento parte de las clases R1 a R6 y del impacto sobre cada aplicación. Se evalúan exposición, criticidad, posibilidad de explotación, soporte disponible y alternativas de reemplazo. La falta de actividad es una señal de mantenimiento que requiere contexto; una licencia no aprobada es un problema de política y cumplimiento, no una vulnerabilidad técnica [1, 47].

La aceptación debe vincularse con una versión determinada, un responsable, controles compensatorios y una fecha de revisión. Una autorización inicial puede dejar de ser válida ante un aviso, un cambio de mantenedor o un incidente de registro. El monitoreo definido en la sección 5.2 relaciona esas novedades con el inventario operativo, en lugar de repetir definiciones de *SBOM* o trazabilidad.

Los indicadores de la organización deben medir cobertura del inventario, decisiones pendientes, excepciones vencidas y tiempos de remediación. Son diferentes de los indicadores experimentales del capítulo 6, que solo evalúan la consistencia de un motor sobre diez escenarios conocidos. Mantener esa separación evita convertir un resultado de laboratorio en una afirmación de reducción de incidentes.

5.11 Fundamentos de la estrategia propuesta

De la comparación se derivan tres decisiones de diseño. Primero, organizar los controles sobre un ciclo común con gobierno transversal. Segundo, conservar para cada control la relación entre amenaza, fundamento, implementación y evidencia. Tercero, expresar por separado su estado de implementación y el alcance de la evaluación. Estas decisiones se concretan en el inventario consolidado y la arquitectura del capítulo 7.

La estrategia propone una adopción gradual: conocer la composición y fijar reglas mínimas; automatizar decisiones y conservar evidencias; después incorporar garantías de construcción, distribución y operación. La *PoC* se ubica en la validación de lógica de políticas y generación de evidencia demostrativa. Las restantes capacidades requieren implementación e investigación adicionales.

6. Enfoque metodológico

Este capítulo describe el camino seguido para transformar los conceptos y referencias analizados en una estrategia de implementación verificable. La metodología se estructuró de manera que cada decisión de diseño pudiera relacionarse con los objetivos de la investigación, con una práctica reconocida y con una evidencia observable en la prueba de concepto. El propósito no fue crear un nuevo *framework*, sino integrar controles existentes en una secuencia coherente, adaptable y orientada al riesgo.

6.1 Tipo y enfoque de investigación

La investigación es de carácter aplicado, dado que busca resolver un problema concreto de la ingeniería de *software*: la incorporación y gestión de dependencias externas sin controles consistentes a lo largo del ciclo de vida. Su finalidad es propositiva, porque a partir de conocimientos existentes se construye una estrategia de implementación que organiza prácticas, responsabilidades, decisiones y evidencias. A su vez, posee un alcance descriptivo, ya que caracteriza los riesgos de la cadena de suministro y examina la cobertura ofrecida por distintos marcos, estándares y guías.

El enfoque predominante es cualitativo y documental. Se analizan referencias normativas y técnicas para identificar conceptos comunes, diferencias de alcance y mecanismos de control complementarios. Este análisis se combina con una validación técnica acotada mediante una prueba de concepto. La *PoC* no pretende demostrar eficacia estadística ni certificar el cumplimiento integral de un estándar, sino comprobar que los componentes principales de la estrategia pueden traducirse en reglas automatizables, decisiones explícitas y evidencias auditables [1, 2, 28].

6.2 Diseño de la investigación

El diseño se organizó en cuatro fases relacionadas entre sí. La primera consistió en relevar el problema, las amenazas y los puntos de exposición asociados a las dependencias. La segunda comprendió el análisis comparativo de *NIST SSDF*, *OWASP SCVS*, *OWASP SAMM*, *SLSA*, *OpenSSF Scorecard*, las recomendaciones sobre *SBOM* y otras guías presentadas en el marco teórico. La tercera fase sintetizó los controles identificados en una estrategia de implementación distribuida a lo largo del ciclo de vida. La cuarta fase evaluó una selección de esos controles en un entorno controlado.

La secuencia adoptada permite mantener trazabilidad entre problema, referencia, control, evidencia y resultado. Los lineamientos de gestión del riesgo se utilizaron para distinguir entre controles preventivos, detectivos y correctivos, y para definir cuándo una condición debía bloquear el flujo, generar una advertencia o requerir una aceptación explícita [47]. De esta ma-

nera, el diseño conecta el análisis conceptual con una demostración técnica sin extender las conclusiones más allá del alcance definido.

6.3 Pregunta de investigación

¿Cómo pueden integrarse referencias existentes en una estrategia trazable para gestionar los riesgos de paquetes de software directos y transitivos, y qué parte de sus controles puede demostrarse mediante una prueba de concepto local con escenarios de laboratorio y herramientas reales?

Esta pregunta se descompone en tres interrogantes operativos. En primer lugar, qué riesgos y decisiones deben contemplarse desde la selección de un componente hasta su retiro. En segundo lugar, qué aportes específicos realizan las referencias analizadas y por qué su combinación resulta más adecuada que la adopción aislada de una de ellas. Finalmente, qué controles pueden demostrarse en una *PoC* acotada y qué evidencias permiten determinar si dichos controles actuaron de acuerdo con la política definida.

6.4 Unidad de análisis y alcance

La unidad de análisis está constituida por las prácticas, controles y evidencias aplicables a la gestión segura de dependencias. No se evalúa un producto comercial específico ni se realiza una comparación exhaustiva de herramientas. El foco se ubica en capacidades tales como inventario, evaluación de vulnerabilidades, control de origen, reproducibilidad, integridad de artefactos, trazabilidad, monitoreo y respuesta.

La estrategia se formula para paquetes distribuidos mediante gestores. La *PoC* utiliza *Node.js*, *npm* y *Trivy*, con proyectos de laboratorio para diez escenarios, principalmente del ecosistema *npm*, y un caso *Maven/Log4j*. Instala paquetes seleccionados con scripts deshabilitados, consulta fuentes externas y conserva manifiestos, lockfiles e informes. No constituye una aplicación ni una integración productiva, y no valida otros gestores.

6.5 Criterios de selección de marcos y estándares

Las referencias se seleccionaron por su relación con el desarrollo seguro y los componentes; la autoridad de su entidad responsable; la disponibilidad de documentación; la posibilidad de traducir sus aportes a prácticas observables; su complementariedad; y su utilidad para trazabilidad y evaluación. Se fijan las ediciones *SSDF* 1.1, *SAMM* 2, *SCVS* 1.0 y *SLSA* 1.0 para mantener reproducible el análisis. Para *ISO* se utiliza el alcance conceptual citado, sin atribuir una evaluación exhaustiva de cláusulas.

La aplicación de estos criterios explica la combinación adoptada. *NIST SSDF* aporta una estructura general para integrar seguridad al ciclo de vida; *SCVS* profundiza la verificación de componentes; *SAMM* contribuye con una perspectiva de madurez; *SLSA* se concentra en la construcción y la procedencia; *OpenSSF Scorecard* ofrece señales sobre proyectos de código

abierto; y las recomendaciones de *CISA*, *NTIA*, *SPDX* y *CycloneDX* fortalecen la visibilidad mediante *SBOM* [1, 2, 7, 28, 29, 36, 38, 39, 41]. Ninguna referencia se interpreta como una solución autosuficiente.

6.6 Procedimiento de análisis comparativo

El análisis se realizó en tres pasos: clasificar cada referencia por su función; identificar sus aportes a Diseño, Implementación, Verificación y Operación, con Gobernanza transversal; y registrar límites y capacidades complementarias. La matriz de la sección 5.8 expresa esa correspondencia analítica mediante aportes concretos, en lugar de asignar puntuaciones de cobertura sin criterios verificables.

Luego se identificaron prácticas recurrentes y relaciones de dependencia entre controles. Por ejemplo, el análisis de vulnerabilidades requiere primero conocer con precisión los componentes y sus versiones; la verificación de procedencia necesita un artefacto identificable mediante un resumen criptográfico; y la respuesta ante incidentes depende de conservar inventarios y evidencias históricas. El procedimiento permitió pasar de una lista de recomendaciones aisladas a un conjunto ordenado de capacidades.

6.7 Procedimiento de construcción de la estrategia

Cada control seleccionado se vinculó con una clase de riesgo, una función del ciclo de vida, una referencia, una implementación posible y una evidencia. El inventario del capítulo 7 consolida esas relaciones y utiliza los estados E, P y N para separar reglas evaluadas, representaciones parciales y capacidades no implementadas. Los responsables se asignan mediante la matriz de roles y el contexto del producto.

La priorización siguió un enfoque basado en riesgo. Las condiciones capaces de introducir una exposición inmediata, como una vulnerabilidad crítica, un origen no autorizado o una alteración del artefacto, se trataron como bloqueantes. Las señales que requieren contexto, como la inactividad de un proyecto o una excepción de licencia, pueden derivarse a revisión manual. Esta separación busca evitar tanto la aceptación automática de riesgos como el bloqueo indiscriminado del desarrollo [47, 48].

6.8 Diseño de la prueba de concepto

La validación principal corresponde al modo real de la *PoC*: crea proyectos de laboratorio, ejecuta comprobaciones de *npm*, consulta metadatos del registro y analiza los proyectos con *Trivy*. No se compara con una línea de base productiva; la función de línea de base existe únicamente en el modo simulado y sus resultados no se mezclan con los del ensayo real.

Se definieron diez escenarios: uno limpio y nueve con condiciones de riesgo. La corrida real utiliza versiones y datos disponibles al momento de ejecución; registra la versión de *Node.js*, *npm*, *Trivy*, la base del escáner y las fuentes consultadas. Los paquetes y hallazgos

pueden cambiar entre corridas. Los comandos de instalación usan `-ignore-scripts`; por lo tanto, los scripts de ciclo de vida no se ejecutan. El modo simulado conserva datos congelados y repeticiones deterministas para probar la lógica, sin constituir evidencia del análisis real.

6.9 Criterios e indicadores de evaluación

La evaluación del ensayo real comprueba la concordancia entre las decisiones esperadas y obtenidas, que los nueve escenarios riesgosos no sean permitidos y que el escenario limpio se permita sin hallazgos. También verifica que cada escenario conserve referencias a sus evidencias. Al tratarse de una corrida con datos externos variables, no se calcula repetibilidad entre ejecuciones ni se interpreta la concordancia como una medida estadística de eficacia. La Tabla 5 formaliza los indicadores y sus criterios de aceptación.

Tabla 5: Indicadores utilizados para evaluar la prueba de concepto

Indicador	**Definición operativa**	**Criterio de aceptación**
Cobertura de detección	Escenarios riesgosos bloqueados o derivados a revisión sobre el total de escenarios riesgosos	9 de 9 escenarios
Exactitud de decisión	Decisiones reales coincidentes con las esperadas sobre el total de escenarios	10 de 10 escenarios
Falsos positivos	Escenarios limpios con hallazgos sobre el total de escenarios limpios	0 de 1 escenario
Reproducibilidad	Comparación de decisiones y evidencias entre distintas corridas reales	No se mide; las fuentes externas pueden cambiar
Completitud de evidencia	Evidencias asociadas a cada escenario según los resultados registrados	Evidencia registrada para cada escenario

Estos criterios se eligieron por corresponder a la finalidad demostrativa de la *PoC*. No se incluyeron mediciones de rendimiento, escalabilidad o costo operativo, ya que un conjunto de diez escenarios locales no permite producir conclusiones representativas sobre esas dimensiones.

6.10 Limitaciones metodológicas

La prueba utiliza un conjunto pequeño construido para ejercitar controles definidos previamente. La concordancia observada no estima sensibilidad, especificidad ni eficacia sobre una población. El registro y la base de vulnerabilidades consultados son reales, pero el conjunto de escenarios no es una muestra aleatoria ni independiente de las reglas. La corrida no construye ni despliega un producto, no prueba un pipeline organizacional y no representa todos los paquetes transitivos de una aplicación.

Además, la investigación no mide cambios culturales, costos de adopción, tiempos de remediación en producción ni comportamiento de usuarios. Tampoco demuestra conformidad completa con *NIST SSDF*, *SCVS*, *SLSA* u otro estándar. La *PoC* verifica un subconjunto de controles en escenarios de laboratorio mediante herramientas reales; estas comprobaciones no

equivalen a su operación continua en una organización. Estas limitaciones se incorporan explícitamente a la discusión para evitar interpretar los resultados como una certificación o como una garantía absoluta de seguridad.

7. Estrategia de implementación para la gestión segura de dependencias

La estrategia propuesta organiza prácticas existentes en un flujo de decisiones que abarca desde la necesidad de incorporar un componente hasta su retiro. Su valor no reside en agregar una nueva colección de requisitos, sino en articular referencias complementarias y asignar a cada control un momento, un responsable y una evidencia. La estrategia puede adaptarse al nivel de riesgo, al tamaño de la organización y al grado de madurez de sus procesos.

7.1 Principios rectores

El primer principio es la gestión basada en riesgo. No todas las dependencias poseen la misma criticidad ni todos los sistemas presentan el mismo impacto potencial; por ello, los umbrales y niveles de aprobación deben considerar exposición, datos procesados, disponibilidad requerida y posibilidad de explotación. El segundo principio es la visibilidad: una organización no puede evaluar ni responder sobre componentes que desconoce. El inventario y el *SBOM* constituyen la base de los controles posteriores [1, 7].

Los demás principios son trazabilidad, reproducibilidad, mínimo privilegio, separación de funciones, automatización responsable y mejora continua. La automatización debe acelerar decisiones repetitivas, pero conservar mecanismos de excepción y responsabilidad humana. La trazabilidad exige registrar quién aprobó, qué regla se aplicó y qué evidencia respaldó la decisión. La reproducibilidad busca que una misma entrada genere el mismo conjunto de dependencias y artefactos. En conjunto, estos principios reducen decisiones informales y facilitan la auditoría [2, 29, 44].

7.2 Gobernanza, roles y responsabilidades

La Tabla 6 resume los roles, responsabilidades y evidencias esperadas. La gobernanza comienza con un patrocinador responsable de aprobar la política y resolver riesgos que exceden la autoridad técnica. El área de seguridad define criterios mínimos, umbrales y requisitos de evidencia. Arquitectura analiza la adecuación de los componentes y evita duplicaciones innecesarias. Los equipos de desarrollo justifican la incorporación, mantienen el manifiesto y atienden las remediaciones. El equipo de plataforma integra los controles en repositorios y *pipelines*.

Legal o cumplimiento interviene ante licencias, obligaciones contractuales y proveedores. Operaciones mantiene el inventario desplegado y colabora con respuesta a incidentes. La decisión de excepción debe tener propietario, fundamento, alcance y fecha de expiración. Esta distribución evita que la seguridad de las dependencias recaiga exclusivamente sobre el desa-

rrollador y se alinea con el enfoque transversal de *SSDF* y *DevSecOps* [2, 45, 46].

Tabla 6: Responsabilidades principales dentro de la estrategia

Rol	Responsabilidad principal	Evidencia esperada
Patrocinador o dueño del riesgo	Aprobar política y aceptar riesgos de alto impacto	Política y aceptación formal
Seguridad	Definir reglas, umbrales y tratamiento de hallazgos	Reglas versionadas e informes
Arquitectura	Evaluar adecuación y alternativas de componentes	Registro de decisión técnica
Desarrollo	Declarar, justificar, actualizar y probar dependencias	Manifiesto, archivo de bloqueo y solicitud de cambio
Plataforma	Automatizar controles y proteger la construcción	Configuración del *pipeline* y registros
Legal o cumplimiento	Evaluar licencias y obligaciones externas	Dictamen o lista de licencias permitidas
Operaciones y respuesta	Mantener inventario desplegado y coordinar incidentes	Inventario, alertas y registro de incidente

7.3 Ciclo de vida de las dependencias

La estrategia utiliza el modelo de la sección 5.6. Diseño comprende necesidad, alternativas y requisitos; Implementación comprende declaración, resolución y construcción; Verificación comprende evaluación y autorización; Operación comprende monitoreo, respuesta, actualización y retiro. Las políticas, los responsables, las excepciones y los indicadores se mantienen en Gobernanza transversal. Las actividades se reiteran ante cambios, sin imponer fases rígidamente consecutivas.

El seguimiento continúa después de la aprobación inicial. Una versión aceptable puede volverse vulnerable, quedar abandonada o cambiar sus condiciones de distribución. Por eso, el *SBOM* y el inventario desplegado deben relacionarse con fuentes de inteligencia y con procesos de actualización. El retiro también forma parte del ciclo: eliminar un componente implica confirmar que no permanezca en ramas activas, imágenes, artefactos históricos reutilizables o entornos productivos.

7.4 Arquitectura de referencia e implementación mínima

La Figura 4 muestra una arquitectura lógica aplicable a un equipo que incorpora paquetes de terceros. Los productos concretos pueden sustituirse si conservan entradas, salidas y responsabilidades. El repositorio mantiene código, manifiestos, archivos de bloqueo y políticas. Un gestor o proxy de paquetes concentra las fuentes autorizadas. El pipeline obtiene esos insumos, analiza los componentes y consulta el motor de políticas antes de promover un resultado.

El *SCA* recibe la composición resuelta y los avisos de vulnerabilidad; el motor de polí-

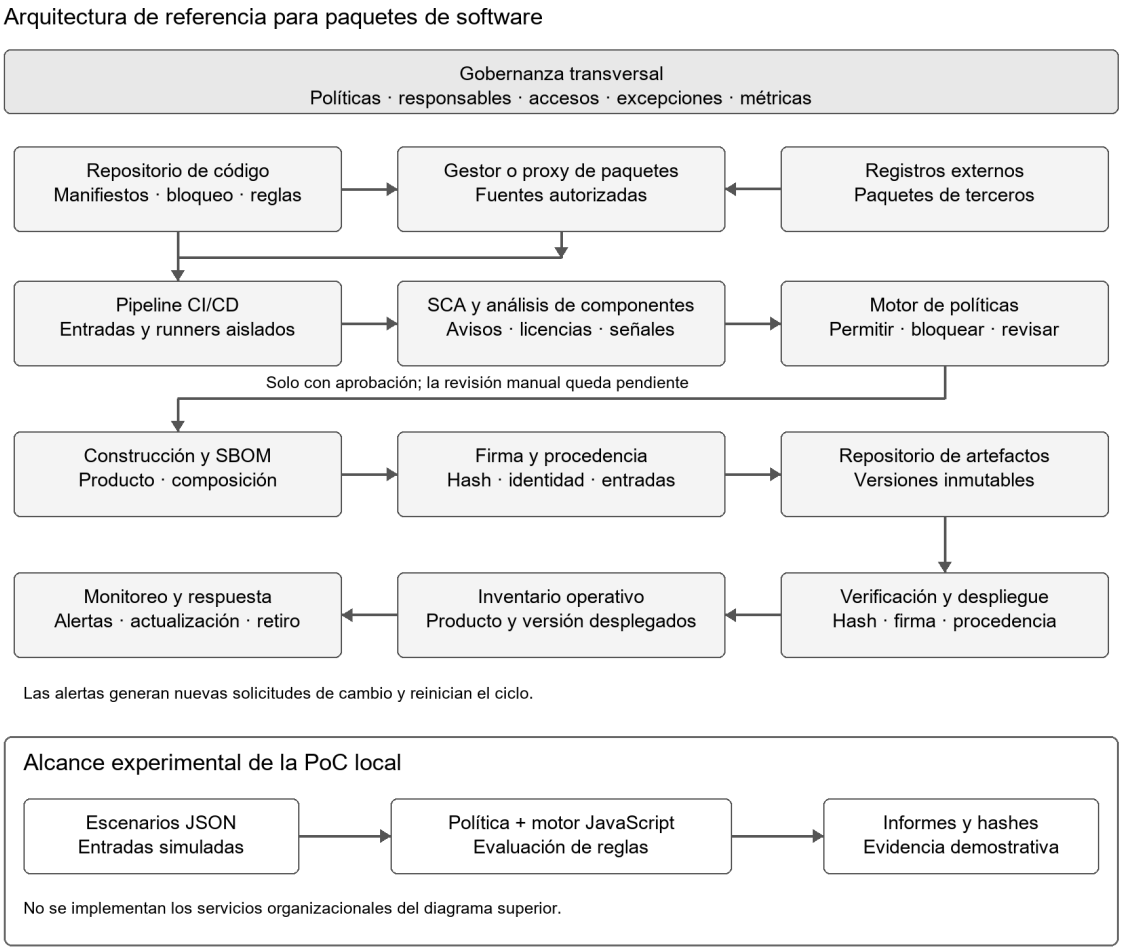


Figura 4: Arquitectura de referencia y representación del modo simulado separado de la *PoC*. Fuente: elaboración propia con base en [1, 2, 29, 45].

ticas combina esos hallazgos con origen, licencias, mantenimiento y reglas de aprobación. El generador de *SBOM* produce el inventario del producto. La construcción aprobada genera un artefacto identificado por hash y una declaración de procedencia; una identidad autorizada firma las evidencias que correspondan. El repositorio de artefactos conserva versiones inmutables y el despliegue verifica hash, firma, procedencia y decisión antes de promoverlas.

Las fronteras de confianza son explícitas: el registro público es una fuente externa; el proxy no garantiza que todo lo almacenado sea benigno; el runner debe aislar código no confiable; los permisos de firma y publicación se separan de las etapas de análisis; y el entorno productivo acepta solo artefactos aprobados. Operación vincula el inventario desplegado con alertas y abre solicitudes de actualización que reinician el ciclo. Gobernanza establece las reglas, revisa las excepciones y controla los accesos en todos los componentes.

Como implementación mínima se propone un repositorio Git protegido, *npm* con archivo de bloqueo y registro autorizado o proxy, un ejecutor CI, un analizador *SCA*, un generador de *SBOM* y reglas versionadas con conservación de informes. Ejemplos sustituibles son GitHub o GitLab para repositorio y CI, *npm* audit u OSV-Scanner para avisos, un generador *CycloneDX* para inventario y *OPA* o un motor propio para reglas. Estas herramientas son opciones de diseño, no herramientas evaluadas por la *PoC* [44, 50, 51, 52].

La aceptación mínima requiere que cada cambio de dependencias produzca un informe asociado a una revisión, que los bloqueos detengan la promoción y que una revisión manual permanezca pendiente hasta su resolución. La adopción posterior incorpora repositorio de artefactos, firma y procedencia verificable, y monitoreo del inventario operativo. No se considera completada una capacidad porque una herramienta esté instalada: debe existir evidencia de que el control actúa sobre el flujo real.

En la versión publicada de la *PoC*, `poc_gestion_dependencias/src/real-runner.js` crea proyectos de laboratorio, consulta el registro *npm*, ejecuta instalaciones con scripts deshabilitados y llama a *Trivy*; `poc_gestion_dependencias/config/policy.json` contiene los umbrales y criterios. Los resultados, proyectos y evidencias se guardan en `poc_gestion_dependencias/output/real-run/`. El modo simulado independiente conserva los escenarios JSON y las pruebas deterministas del motor; sus salidas no se mezclan con la corrida real. No se implementaron un proxy, un pipeline de CI alojado, un repositorio de artefactos, un servicio de firma ni monitoreo productivo.

7.5 Inventario consolidado de controles

El inventario reúne los controles de la estrategia y permite distinguir su fundamento de su implementación. Las funciones D, I, V, O y G corresponden a la sección 5.6; los riesgos R1 a R6, a la sección 5.3.1. El detalle de la implementación, los escenarios de la *PoC* asociados, las evidencias generadas y las limitaciones se presentan en el Anexo A (Tabla A.1).

Tabla 7: Inventario consolidado de controles de la estrategia

ID	Control	Riesgo y función	Fundamento
C01	Política y responsables	R1–R6; G	*SSDF* PO.1/PO.2; *SAMM* Governance [2, 28]
C02	Evaluación de necesidad	R1, R2, R5, R6; D	*SCVS* 5; *SAMM* Design [1, 28]
C03	Fuentes autorizadas	R2, R3; I/V	*SCVS* 4.1/4.2/6.1 [1]
C04	Versiones y bloqueo	R1, R2, R4; I/V	*SCVS* 3.1/5.6 [1]
C05	Inventario efectivo	R1–R6; I/O	*SCVS* 1.1/1.7 [1]
C06	Vulnerabilidades conocidas	R1; V/O	*SSDF* RV.1/RV.2; *SCVS* 5.4 [1, 2]
C07	Código y scripts no aprobados	R2, R4; I/V	*SCVS* 4.18; *CIS* [1, 30]
C08	Integridad de contenido	R4; V	*SSDF* PS.2; *SCVS* 4.13/4.14 [1, 2]
C09	Mantenimiento y soporte	R5; D/O	*SCVS* 5.7/5.8; *Scorecard* [1, 41]
C10	Licencias	R6; D/V	*SCVS* 5.12 [1]
C11	Decisiones automatizadas	R1–R6; V/G	*SSDF* PO.4; políticas como código [2, 44]
C12	*SBOM* válido y completo	R1, R5, R6; I/V/O	*SCVS* 2; *NTIA* [1, 36, 39]
C13	Procedencia verificable	R4; I/V	*SLSA* Build; *SCVS* 6 [1, 29]
C14	Firma y comprobación	R3, R4; I/V	*SLSA* Build L2; *SCVS* 2.4–2.6 [1, 29]
C15	Identidades y repositorio	R3, R4; I/G	*SCVS* 4.3–4.5; *CIS* [1, 30]

Continúa en la siguiente página

Tabla 7 – continuación

ID	Control	Riesgo y función	Fundamento
C16	Entorno de construcción	R2, R4; I	*SCVS* 3; *SLSA* Build L3 [1, 29]
C17	Promoción e inmutabilidad	R3, R4; I/V/O	*SSDF* PS.3; *SLSA* [2, 29]
C18	Excepciones temporales	R1–R6; G/V	*ISO* 31000; *SAMM* [28, 47]
C19	Monitoreo y respuesta	R1, R2, R3, R5; O	*SSDF* RV; *SCVS* 1.6 [1, 2]
C20	Actualización y retiro	R1, R5, R6; O/D	*SCVS* 3.20/3.21/5.7/5.8 [1]
C21	Capacitación y mejora	R1–R6; G	*SAMM* Governance [28]
C22	Conservación de evidencia	R1–R6; G/V	*SCVS* 3.13/3.14; *SSDF* PS.3 [1, 2]

La implementación concreta, los escenarios de la *PoC* (S0–S9), las evidencias generadas y las limitaciones por control se detallan en la Tabla A.1 del Anexo A. Este inventario es la referencia para interpretar los resultados del capítulo 9: ejecutar un subconjunto de controles en un ensayo local no equivale a implementar la capacidad completa en una organización.

Para adoptar el inventario, el responsable de cada producto debe registrar aplicabilidad, implementación concreta, evidencia, estado y excepción vigente. Si cambia una regla o una fuente, debe conservarse su versión. La correspondencia con fuentes sirve para justificar la selección; demostrar conformidad exige revisar todos los requisitos aplicables de la edición citada.

7.6 Preparación y definición de políticas

La organización debe clasificar aplicaciones y definir para cada clase un nivel de exigencia. La política mínima establece repositorios autorizados, uso obligatorio de archivos de bloqueo, umbrales de vulnerabilidad, licencias permitidas, tratamiento de *scripts* de instalación, frecuencia de actualización, evidencias requeridas y autoridades de excepción. Estas reglas deben ser comprensibles, versionadas y revisadas ante incidentes o cambios regulatorios.

Una política efectiva diferencia entre bloqueos, advertencias y revisiones manuales. Por ejemplo, una vulnerabilidad crítica explotable sin mitigación puede bloquear de forma inme-

diata, mientras que una dependencia con baja actividad pero sin vulnerabilidades conocidas puede requerir análisis de contexto. También se deben definir plazos de remediación según severidad y criticidad. *SCVS* destaca que la aceptación del riesgo no puede resolverse únicamente mediante herramientas: requiere decisiones organizacionales explícitas [1].

7.7 Selección e incorporación segura de dependencias

Antes de incorporar una dependencia se debe comprobar que su función no pueda resolverse con capacidades ya aprobadas o con una alternativa de menor complejidad. La evaluación considera procedencia, popularidad sin asumir que esta equivale a seguridad, actividad de mantenedores, frecuencia de versiones, tiempo de respuesta a vulnerabilidades, profundidad del árbol transitivo, licencias, documentación y compatibilidad técnica. *OpenSSF Scorecard* puede aportar señales automatizadas sobre prácticas del proyecto, pero sus resultados deben interpretarse junto con el contexto [41].

La solicitud de incorporación debe registrar finalidad, versión propuesta, propietario interno, alternativa evaluada y resultado de los controles. Las dependencias directas requieren una justificación clara; las transitivas deben ser visibles mediante el archivo de bloqueo y el *SBOM*. Cuando el riesgo supera el umbral, las opciones son seleccionar otra biblioteca, aplicar una mitigación temporal documentada o elevar la decisión al dueño del riesgo.

7.8 Configuración segura del repositorio

El repositorio constituye la primera barrera contra cambios no autorizados. Se recomienda proteger ramas principales, exigir revisión por pares, restringir la modificación de archivos sensibles y aplicar autenticación multifactor a cuentas con privilegios. Los manifiestos, archivos de bloqueo, configuraciones de registro, reglas de política y definiciones del *pipeline* deben quedar bajo control de versiones. Los cambios en estos archivos requieren revisores competentes y no deben aprobarse de forma automática.

También se deben limitar tokens y secretos, fijar por versión o resumen las acciones utilizadas por el *pipeline* y conservar registros de administración. La firma de *commits* puede aportar autenticidad, pero no sustituye la revisión ni el control de permisos. *OpenSSF* y *CIS* recomiendan combinar protección de ramas, revisión, mínima exposición de credenciales y monitoreo de cambios para reducir el riesgo de compromiso del repositorio [30, 32].

7.9 Gestión de entornos diferenciados

Los entornos de desarrollo, integración y producción deben mantener fronteras claras. Las credenciales de producción no deben estar disponibles en estaciones de trabajo ni en ejecuciones de prueba. Los registros internos pueden separar componentes evaluados de paquetes aún no aprobados y actuar como punto de control entre los gestores públicos y los procesos de construcción.

La promoción debe realizarse sobre artefactos inmutables, no mediante una nueva compilación por ambiente. El mismo artefacto verificado en integración debe avanzar a producción acompañado por su resumen, *SBOM* y procedencia. Esta práctica reduce la posibilidad de que el resultado desplegado difiera del evaluado y se alinea con las garantías progresivas propuestas por *SLSA* [29].

7.10 Validación local y revisión colaborativa

Antes de enviar un cambio, el desarrollador debe actualizar el manifiesto y el archivo de bloqueo de forma conjunta, revisar las dependencias transitivas agregadas y ejecutar las verificaciones disponibles. Cuando el gestor lo permita, la instalación inicial debe evitar *scripts* de ciclo de vida hasta que estos hayan sido inspeccionados. Las diferencias del archivo de bloqueo merecen revisión específica, porque un cambio pequeño en el manifiesto puede incorporar numerosos componentes transitivos.

La solicitud de cambio debe mostrar el resultado de las verificaciones, la justificación y el impacto esperado. La revisión colaborativa no se limita a la corrección funcional: debe comprobar origen, versión, licencia, mantenimiento y necesidad. Esta etapa conserva el criterio humano antes de los controles automatizados y disminuye el costo de corregir una decisión una vez que la dependencia ya fue promovida.

7.11 Automatización de controles en *CI/CD*

Los controles se integran como etapas ordenadas del *pipeline*: validación de manifiestos, instalación reproducible, análisis de componentes, evaluación de políticas, pruebas, generación de *SBOM*, construcción, cálculo de resumen, atestación y publicación. Una falla bloqueante debe detener las etapas posteriores y conservar un informe legible. Las advertencias deben crear una acción o aprobación explícita, no quedar ocultas entre registros técnicos.

Las políticas como código permiten versionar umbrales y aplicar decisiones homogéneas. *OPA*, por ejemplo, puede evaluar datos estructurados y devolver códigos de salida que detienen una ejecución [44]. La estrategia no depende de ese producto: cualquier motor capaz de procesar entradas, reglas y resultados auditables puede cumplir la función. Es importante fijar versiones de herramientas y reglas para que el control también sea reproducible.

7.12 Generación y gestión del *SBOM*

El *SBOM* debe generarse durante la construcción a partir de la resolución efectiva de dependencias, no de una lista mantenida manualmente. Debe incluir componentes directos y transitivos, versiones, identificadores consistentes, relaciones y metadatos suficientes para relacionarlo con un artefacto. *SPDX* y *CycloneDX* ofrecen formatos interoperables; la elección debe responder a los consumidores y procesos de la organización [36, 38, 39].

Cada nueva versión del producto debe producir un *SBOM* asociado a su resumen cripto-

gráfico. El documento debe almacenarse junto con el artefacto o en un repositorio consultable, conservar historial y someterse a control de acceso. La comparación entre versiones permite detectar incorporaciones inesperadas. Para la operación, el *SBOM* debe conectarse con avisos de vulnerabilidad y, cuando corresponda, con información *VEX* que indique el estado de afectación sin eliminar la evidencia original.

7.13 Protección de la compilación y los artefactos

La compilación debe ejecutarse en entornos limpios, efímeros y con privilegios mínimos. Las herramientas, imágenes base y acciones deben estar fijadas por versión o resumen. La descarga dinámica de componentes no declarados debe evitarse, y las cachés deben tratarse como entradas no confiables cuya integridad se verifica antes del uso. Los secretos solo se habilitan en la etapa que los necesita y nunca se incorporan al artefacto.

El resultado se identifica mediante un resumen criptográfico y se almacena en un repositorio con controles de escritura y retención. No debe modificarse después de su publicación. *NIST SSDF* propone proteger todas las formas de código y preservar la integridad de las versiones; *SLSA* complementa esta práctica al fortalecer el entorno de construcción y la evidencia sobre cómo se produjo el artefacto [2, 29].

7.14 Firma, procedencia y distribución

La procedencia describe el proceso y las entradas de un artefacto, según la sección 5.5. Una firma puede vincular la declaración con una identidad verificable; una atestación sin firma no aporta esa garantía. El consumidor debe comprobar identidad, parámetros esperados y coincidencia entre el hash declarado y el artefacto recibido. La estrategia exige esa verificación antes de promover; la *PoC* solo representa parte de los datos.

La estrategia propone una adopción gradual. En una primera etapa se conservan resumen, revisión de origen y metadatos de construcción. En una etapa intermedia la plataforma genera y firma la procedencia. En niveles avanzados se utilizan entornos endurecidos, separación del material de firma y verificación obligatoria antes de la promoción. Esta progresión es consistente con los niveles del *Build Track* de *SLSA* [29].

7.15 Despliegue seguro y monitoreo continuo

Antes del despliegue se debe verificar que el artefacto corresponda al aprobado, que su procedencia provenga de una plataforma confiable y que el *SBOM* esté disponible. La promoción se realiza por resumen inmutable. El inventario operativo registra qué versión del producto y qué conjunto de componentes se encuentran en cada entorno.

El monitoreo correlaciona ese inventario con nuevas vulnerabilidades, cambios de mantenimiento, incidentes de proveedores y alertas de repositorios. La reevaluación debe ser continua, porque el riesgo cambia aunque el código permanezca sin modificaciones. Las alertas

requieren un propietario, una prioridad y un plazo; de lo contrario, la visibilidad no se convierte en reducción efectiva del riesgo [7, 23, 35].

7.16 Actualización, sustitución y retiro de dependencias

Las actualizaciones se clasifican en rutinarias, de seguridad y de emergencia. Las rutinarias siguen una cadencia que evita acumular saltos demasiado grandes. Las de seguridad se priorizan según severidad, explotabilidad, exposición del sistema y disponibilidad de mitigaciones. Las de emergencia pueden utilizar un flujo abreviado, pero deben conservar pruebas, aprobación y evidencia.

Cuando una dependencia queda abandonada, pierde compatibilidad, cambia su licencia o supera el riesgo aceptable, debe definirse un plan de sustitución. El retiro incluye eliminarla del manifiesto y del archivo de bloqueo, regenerar el *SBOM*, reconstruir el artefacto y verificar que no permanezca transitivamente. La excepción temporal debe tener responsable, controles compensatorios y fecha de vencimiento.

7.17 Gestión de incidentes

Ante la divulgación o sospecha de compromiso de una dependencia, la primera tarea es determinar exposición. El inventario y los *SBOM* permiten localizar aplicaciones, versiones y ambientes afectados. Luego se contiene el riesgo mediante bloqueo de versiones, suspensión de despliegues, revocación de credenciales o aislamiento de artefactos. La remediación puede requerir actualizar, sustituir o reconstruir desde una revisión confiable.

El proceso debe preservar evidencias, comunicar a propietarios y registrar decisiones. Después de la recuperación se revisan las causas y se ajustan reglas, umbrales o procedimientos. El incidente de *Axios* presentado anteriormente ilustra la importancia de disponer de una respuesta que no dependa únicamente de que cada equipo recuerde dónde utiliza un paquete [12]. Una estrategia madura convierte el aprendizaje del incidente en una mejora verificable.

7.18 Capacitación y concientización

La capacitación debe adaptarse a cada rol. Los desarrolladores necesitan comprender archivos de bloqueo, dependencias transitivas, *scripts* de instalación y revisión de cambios. Los responsables de seguridad deben interpretar resultados de *SCA* y establecer umbrales. Plataforma necesita proteger *pipelines*, secretos y artefactos. Arquitectura, compras y legal requieren criterios para evaluar alternativas, proveedores y licencias.

La formación debe utilizar ejemplos cercanos y ejercicios de respuesta, no limitarse a una comunicación anual. Los indicadores de aprendizaje pueden incluir calidad de las solicitudes, reducción de excepciones vencidas y tiempo de atención. *DevSecOps* enfatiza que la seguridad se integra al trabajo cotidiano cuando las personas comprenden el propósito de los controles y reciben retroalimentación oportuna [43, 45].

7.19 Niveles de adopción de la estrategia

Se proponen tres niveles de adopción para permitir una implementación gradual. El nivel inicial establece visibilidad y controles mínimos. El nivel gestionado agrega políticas como código, responsabilidades formales y evidencias consistentes. El nivel avanzado incorpora procedencia verificable, entornos endurecidos, monitoreo organizacional y mejora basada en métricas. Los niveles no representan una certificación; son una guía de priorización adaptable. La Tabla 8 resume las capacidades y resultados esperados por nivel.

Tabla 8: Niveles propuestos para la adopción de la estrategia

Nivel	**Capacidades principales**	**Resultado esperado**
Inicial	Inventario, archivo de bloqueo, análisis de vulnerabilidades, repositorios autorizados y responsables definidos	Conocer componentes y detener riesgos evidentes
Gestionado	*SBOM* por versión, políticas como código, tratamiento de excepciones, actualización programada y evidencias centralizadas	Aplicar decisiones homogéneas y auditables
Avanzado	Construcción endurecida, procedencia firmada, verificación previa al despliegue, inventario organizacional y métricas	Aumentar confianza de extremo a extremo y mejorar de forma continua

La organización puede alcanzar niveles diferentes según la criticidad de cada producto. Un sistema interno de bajo impacto puede mantener controles iniciales, mientras una aplicación expuesta o regulada requiere capacidades avanzadas. Esta flexibilidad evita imponer el mismo costo a todos los proyectos y coincide con el enfoque incremental de *SCVS* y *SAMM* [1, 28].

7.20 Indicadores y mejora continua

Los indicadores deben relacionarse con resultados y no solo con cantidad de herramientas. Se recomienda medir porcentaje de aplicaciones con *SBOM* actualizado, cobertura de dependencias directas y transitivas, vulnerabilidades por severidad, tiempo medio de remediación, excepciones activas y vencidas, dependencias sin mantenimiento, compilaciones con procedencia y despliegues verificados.

Las tendencias deben analizarse por criticidad y equipo para identificar causas, no para penalizar. Un aumento inicial de vulnerabilidades puede reflejar mejor visibilidad. Las revisiones periódicas deben convertir los hallazgos en acciones sobre políticas, automatización, capacitación o arquitectura. La mejora continua cierra el ciclo entre evidencia operativa y decisiones de gobierno, y permite que la estrategia evolucione junto con las amenazas y los procesos [28, 47].

8. Prueba de concepto

La prueba de concepto traduce una parte representativa de la estrategia en un flujo ejecutable. Se priorizaron controles que pueden evaluarse con proyectos de laboratorio, fuentes externas y herramientas reales, y producir una decisión observable con evidencia asociada. La *PoC* funciona como demostración de viabilidad: no sustituye una prueba piloto en producción ni pretende representar la totalidad de controles organizacionales.

El proyecto mantiene un repositorio público en GitHub [53]. La revisión concreta de código y la ejecución que sustentan los resultados se identifican en el Anexo A.

8.1 Objetivo de la prueba de concepto

El objetivo es comprobar que un subconjunto de controles puede aplicarse a dependencias de laboratorio mediante una secuencia automatizada que identifica condiciones de riesgo, aplica una política y conserva evidencia para permitir, bloquear o derivar a revisión manual. La corrida real no implementa una comparación con un flujo básico ni detiene una promoción productiva.

La validación se concentra en nueve escenarios riesgosos que representan vulnerabilidades críticas y altas, origen no autorizado, inconsistencia del archivo de bloqueo, ejecución de scripts de instalación, alteración del artefacto, falta de mantenimiento, licencia no permitida y una combinación de amenazas. También se incorpora un escenario limpio para comprobar que el control no interfiere con una entrada que cumple la política.

8.2 Descripción del entorno controlado

El entorno utiliza *Node.js*, *npm* y *Trivy*. El ejecutor genera proyectos de laboratorio y, en los casos aplicables, instala paquetes con `npm ci -ignore-scripts`. S1 contiene un proyecto *Maven* con *Log4j* 2.14.1. La política versionada fija registros permitidos, umbrales de vulnerabilidad, antigüedad máxima de mantenimiento, licencias aceptadas y deshabilitación de scripts de ciclo de vida.

La corrida real consulta el registro *npm* y la base de vulnerabilidades de *Trivy*, y descarga paquetes fijados para los proyectos de laboratorio. Los scripts de instalación se deshabilitan; S4 comprueba que un `postinstall` no se ejecute. S2 y S9 contienen una URL de registro no autorizada en la configuración o el lockfile; la política la identifica y no intenta descargar desde ese origen. Los resultados varían con los metadatos del registro, las versiones seleccionadas y la base del escáner; la corrida conserva esos datos y las referencias originales para su interpretación [35, 48].

8.3 Arquitectura y tecnologías utilizadas

Dentro de `poc_gestion_dependencias/`, el modo real conecta `src/real-runner.js`, la política `config/policy.json`, los proyectos generados y *Trivy*. Guarda `output/real-run/results.json`, `summary.csv`, manifiestos y lockfiles, reportes originales del escáner y evidencias por escenario. Las pruebas automatizadas se ejecutan por separado con `npm test`; incluyen pruebas del modo simulado y del análisis de reportes y reglas. La Figura 4 presenta la arquitectura organizacional propuesta; su recuadro inferior representa únicamente el modo simulado, no el flujo real que produjo los resultados del capítulo 9.

En la corrida real, *npm* resuelve e instala las dependencias de laboratorio, *Trivy* analiza vulnerabilidades y licencias, y el ejecutor aplica reglas complementarias sobre origen, lockfiles, scripts, integridad y mantenimiento. No se genera un *SBOM* en este modo. El modo simulado sí construye un artefacto demostrativo *CycloneDX*; sus limitaciones se describen por separado y no se presentan como salida del escáner [39, 44].

8.4 Línea de base y alcance de la comparación

La corrida real no contiene un flujo de línea de base contra el cual comparar sus decisiones. El modo simulado incorpora una función mínima que acepta los escenarios cuando el manifiesto se considera válido; esta función se utiliza para pruebas deterministas del motor, no como medición de un proceso real ni como resultado empírico de la validación principal.

Por ese motivo, los resultados del capítulo 9 informan las decisiones de la corrida real frente a las decisiones esperadas de la política. No estiman una mejora antes/después ni el desempeño frente a prácticas habituales de una organización.

8.5 Escenarios de riesgo evaluados

Se definieron diez casos: S0 limpio, con la versión `latest` de `chalk` fijada en el lockfile; S1 con `log4j-core:2.14.1`; S2 con registro no autorizado; S3 con manifiesto y lockfile inconsistentes; S4 con un `postinstall`; S5 con un artefacto `lodash` original y una copia alterada; S6 con antigüedad de release superior a 730 días; S7 con licencia no permitida; S8 con vulnerabilidades altas en `path-to-regexp@0.1.7`; y S9 con hallazgos combinados. Las vulnerabilidades de S1 y S8 proceden del reporte real de *Trivy*; su identificación y severidad pueden variar con la base consultada. La Tabla 9 resume las condiciones y decisiones esperadas.

La decisión de tratar S6 y S8 como revisión manual refleja que la falta de actividad y una vulnerabilidad alta requieren contexto. En esta corrida S6 se basó en metadatos del registro para `request`; S8, en los avisos y puntuaciones del reporte de *Trivy*. S1 a S5, S7 y S9 activaron controles bloqueantes de la política.

Tabla 9: Escenarios utilizados en la prueba de concepto

Escenario	Condición evaluada	Decisión esperada
S0	Paquete limpio; instalación con scripts deshabilitados y escaneo *Trivy*	Permitir
S1	`log4j-core:2.14.1`; vulnerabilidades críticas detectadas por *Trivy*	Bloquear
S2	Registro efectivo y URLs del lockfile fuera de la lista autorizada	Bloquear
S3	`npm ci` rechaza el lockfile inconsistente	Bloquear
S4	`postinstall` detectado; instalación sin ejecutar scripts	Bloquear
S5	Integridad publicada de `lodash` frente a artefacto original y copia alterada	Bloquear la copia alterada
S6	Último release de `request` supera 730 días	Revisión manual
S7	*Trivy* detecta la licencia `GPL-3.0-only`	Bloquear
S8	*Trivy* detecta vulnerabilidades altas en `path-to-regexp@0.1.7`	Revisión manual
S9	Vulnerabilidades, registro no autorizado e integridad alterada	Bloquear

8.6 Controles implementados

El ejecutor compara el registro efectivo de *npm* y las URLs de resolución del lockfile con las fuentes aprobadas, y utiliza `npm ci` para comprobar consistencia y reproducibilidad. *Trivy* aporta vulnerabilidades y licencias: los hallazgos `CRITICAL` o con *CVSS* igual o superior a 9 bloquean; los `HIGH` o con puntuación igual o superior a 7 requieren revisión manual. El ejecutor detecta scripts de ciclo de vida, comprueba la antigüedad del release y verifica en S5 la integridad del artefacto con el valor *SHA-512 SRI* publicado por *npm*. La política 2.0.0 define la antigüedad máxima de 730 días, las licencias admitidas y la prohibición de scripts.

La comprobación de integridad se realiza sobre los bytes del tarball descargado y su copia alterada; el valor de referencia es el publicado por *npm*. La antigüedad se calcula a partir de los metadatos de releases del registro. La lista de licencias admitidas comprende MIT, Apache-2.0, BSD-2-Clause y BSD-3-Clause. La decisión prioriza cualquier hallazgo bloqueante sobre las advertencias. Las capacidades parciales y no implementadas constan en el inventario del capítulo 7 y en la Tabla A.1.

8.7 Integración de controles en el pipeline

El ejecutor evalúa los controles aplicables, acumula hallazgos y aplica la precedencia bloquear, revisión manual y permitir. S3 clasifica por separado errores de inconsistencia de lockfile y de red. Las decisiones quedan registradas en results.json y summary.csv junto con referencias a reportes y evidencias. La ejecución no detiene un pipeline de CI/CD ni una promoción productiva; describe un ensayo local y debe distinguirse de la arquitectura organizacional de la Figura 4.

La salida contiene permitir, bloquear o revision_manual. Son resultados de la evaluación local. La corrida finaliza con error si uno de los escenarios no obtiene la decisión esperada o

falla un control de ejecución; una decisión bloqueante esperada no constituye un error. Ejecutar el programa con éxito no autoriza un despliegue. Una integración futura debe detener la promoción ante bloquear y mantener revision_manual pendiente hasta una aprobación registrada.

8.8 Generación de evidencias

La corrida real produce results.json, summary.csv y evidencias específicas: reportes JSON originales y comandos de *Trivy*, inspecciones de registro, resultados de `npm ci`, inspección de scripts, metadatos de mantenimiento y la referencia de integridad para S5. El manifiesto global registra tamaños y hashes SHA-256 de archivos de entrada y salida. El modo real no genera `sbom.cdx.json` ni `provenance.intoto.json`; esos archivos pertenecen únicamente al modo simulado.

La verificación de S5 utiliza la integridad *SHA-512 SRI* publicada por el registro para contrastar los bytes descargados del tarball original y de una copia alterada. El manifiesto de la corrida, por separado, calcula SHA-256 sobre los bytes de los archivos de evidencia; ninguno de esos hashes autentica una identidad ni sustituye una firma. Las evidencias se conservan localmente y no se someten a un repositorio inmutable.

8.9 Procedimiento de ejecución

Se cargó la política 2.0.0 y se verificó la disponibilidad de *Node.js*, *npm* y *Trivy*. El ejecutor preparó los proyectos, consultó el registro y la base de vulnerabilidades, ejecutó las instalaciones con scripts deshabilitados y evaluó los diez escenarios. En el resultado quedaron registradas la fecha de observación, las versiones de herramientas, la versión y fecha de la base de *Trivy*, las decisiones, hallazgos y rutas a las evidencias.

Los resultados se consolidaron en JSON y CSV. La aceptación se determinó por la concordancia de cada decisión con la esperada, por la detección de una condición no permisiva en los nueve escenarios riesgosos y por la ausencia de hallazgos en S0. Como el registro y la base de vulnerabilidades son variables, otra corrida puede arrojar hallazgos diferentes; el informe conserva las fuentes y versiones para contextualizar cada resultado.

8.10 Criterios de aceptación

La corrida se considera satisfactoria si las diez decisiones coinciden con las esperadas, los nueve escenarios riesgosos se bloquean o se derivan a revisión, S0 se permite sin hallazgos, se generan los reportes y evidencias previstos, y no se ejecutan los scripts de ciclo de vida. El criterio de evidencia verifica disponibilidad y trazabilidad de archivos; no exige una firma ni una custodia externa.

Estos criterios validan la ejecución frente al conjunto definido. S9 reúne varias condiciones y algunos escenarios ejercitan más de un control. Aunque se utilizan paquetes y escáner reales, los casos fueron seleccionados deliberadamente para probar la política. Para estimar

capacidad de detección sería necesario ampliar casos limpios y riesgosos, incluir resultados independientes del diseño de reglas, probar fallas de fuentes y ejecutar un piloto con proyectos de una organización.

9. Resultados y discusión

Este capítulo presenta los resultados de una corrida real sobre escenarios de laboratorio y los interpreta en relación con los objetivos. Por ello, el análisis distingue entre las comprobaciones ejecutadas con herramientas reales y las conclusiones que todavía requieren validación en contextos organizacionales.

9.1 Resultados de la corrida real

La corrida registrada el 6 de noviembre de 2026 utilizó *Node.js* v24.22.1, *npm* 11.11.0 y *Trivy* 0.76.2. Consultó el registro *npm* y la base de vulnerabilidades de *Trivy*; el informe conserva la versión y las fechas de actualización y descarga de esa base. Los diez escenarios obtuvieron la decisión esperada.

La corrida incluye controles reales sobre lockfiles, configuración de registro, artefactos descargados, fechas de publicación y reportes del escáner. No contiene una ejecución comparativa de línea de base, por lo que no se estima una diferencia antes/después frente a otra práctica de seguridad.

9.2 Concordancia y distribución de decisiones

Los nueve escenarios riesgosos recibieron una decisión no permisiva: siete fueron bloqueados y dos derivados a revisión manual. S0 fue permitido sin hallazgos. Las diez decisiones coincidieron con las esperadas en esta corrida, con estado PASS para cada escenario.

Estos valores describen la concordancia con las expectativas del conjunto diseñado: 9/9 escenarios riesgosos no permitidos, 10/10 decisiones coincidentes y 1/1 caso limpio permitido sin hallazgos. No son estimaciones de sensibilidad, especificidad ni tasas de error para una población de dependencias.

9.3 Resultados por escenario de riesgo

S1 se bloqueó luego de que *Trivy* reportara vulnerabilidades críticas en `log4j-core:2.14.1`, incluida *CVE*-2021-44228. *S2* se bloqueó por el registro configurado y las URLs no autorizadas del lockfile; *S3*, porque `npm ci` rechazó la inconsistencia; *S4*, por detectar un `postinstall` sin ejecutarlo; y *S5*, porque el tarball alterado no coincidió con la integridad *SHA-512 SRI* publicada por *npm*.

S6 derivó a revisión manual al detectar que el último release de `request` tenía 2460 días de antigüedad a la fecha observada; esta señal no prueba por sí sola abandono. S7 se bloqueó por la licencia `GPL-3.0-only` detectada por *Trivy*. S8 derivó a revisión manual por vulnerabilidades altas con *CVSS* 7,5 en `path-to-regexp@0.1.7`. S9 se bloqueó por hallazgos de vulnerabilidad, origen no autorizado e integridad alterada. S0 produjo reportes sin hallazgos y

quedó permitido.

9.4 Resumen de resultados del ensayo

La corrida muestra que las reglas implementadas convierten resultados de herramientas y verificaciones de laboratorio en decisiones explícitas. La Tabla 10 presenta únicamente lo observado en esta ejecución, sin atribuir los resultados a una comparación con un flujo de referencia productivo.

Tabla 10: Síntesis de resultados de la corrida real

Indicador	**Resultado observado**	**Alcance**
Decisiones coincidentes con lo esperado	10 de 10	Corrida real del 6 nov. 2026
Escenarios riesgosos no permitidos	9 de 9	Siete bloqueados y dos en revisión manual
Escenario limpio	1 permitido sin hallazgos	Un único caso limpio
Análisis de vulnerabilidades y licencias	Reportes originales de *Trivy*	S0–S4, S7–S9
Evidencias	JSON, CSV y manifiesto SHA-256	Conservación local, sin firma
Repetibilidad de resultados reales	No medida	Fuentes externas variables

La POC conserva entradas, versiones y resultados para facilitar la revisión de la corrida. La repetición del ensayo real puede producir otros resultados si cambian los metadatos del registro o la base de vulnerabilidades; las pruebas deterministas del modo simulado son una comprobación técnica distinta y no se usan como resultados del capítulo.

9.5 Evaluación de los indicadores

La corrida obtuvo 9/9 escenarios riesgosos bloqueados o derivados a revisión y 10/10 decisiones coincidentes con los valores esperados. S0 fue permitido sin hallazgos. Estos cocientes resumen exclusivamente los diez escenarios seleccionados; el único caso limpio no permite estimar una tasa general de falsos positivos. Las evidencias y sus hashes permiten inspeccionar esta corrida, pero no se validan como SBOM, atestación firmada ni custodia inmutable.

Los valores deben leerse junto con el diseño experimental. Los escenarios se eligieron para ejercitar reglas conocidas y no constituyen una muestra de paquetes de una población. Por ello, la concordancia observada describe el comportamiento de esta ejecución, no una capacidad universal de detección. Cada decisión puede relacionarse con la política, el hallazgo y la

evidencia guardada.

9.6 Trazabilidad con los marcos y estándares

Las correspondencias de la Tabla 11 deben leerse junto con los estados del inventario consolidado. La corrida utiliza herramientas externas para análisis de vulnerabilidades y licencias; otras capacidades, como inventario SBOM, firma, procedencia y aprobación humana, permanecen parciales o no implementadas. Las referencias fundamentan el diseño sin implicar conformidad integral.

Tabla 11: Trazabilidad de la prueba de concepto con las referencias analizadas

Capacidad demostrada	**Evidencia de la *PoC***	**Referencias relacionadas**
Inventario de componentes	Manifiestos y lockfiles de proyectos; no se genera SBOM real	*SCVS*, *CISA*, *NTIA* y *CycloneDX* [1, 7, 36, 39]
Control de origen y lockfile	Configuración efectiva, URLs resueltas y prueba con `npm ci`	*SCVS* y *OpenSSF* [1, 32]
Evaluación de vulnerabilidades y licencias	Reportes originales de *Trivy*; umbrales de bloqueo y revisión	*NIST SSDF*, *NVD* y *CVSS* [2, 35, 48]
Política automatizada	Decisión por escenario y hallazgos registrados	Reglas *JavaScript*; enfoque de políticas como código [43, 44]
Integridad del artefacto	*SHA-512 SRI* publicado para el tarball original frente a copia alterada	*NIST SSDF* y *SLSA* [2, 29]
Procedencia y firma	No implementadas en el modo real	*SLSA* [29]
Tratamiento del riesgo	Bloqueo, revisión manual o aceptación	*ISO* 31000 y *OWASP SAMM* [28, 47]

La matriz evidencia la complementariedad observada en el marco teórico. La ejecución utiliza informes de vulnerabilidades y licencias y verificaciones específicas de lockfile, origen, scripts, mantenimiento e integridad. El *SBOM*, la firma y la procedencia siguen siendo capacidades propuestas para una integración futura y no salidas de la corrida real.

9.7 Cumplimiento de los objetivos de investigación

El análisis de riesgos se materializó en escenarios asociados a origen, vulnerabilidad, instalación, integridad, mantenimiento y licencia. La comparación de marcos permitió identificar aportes diferenciados y prácticas comunes. A partir de esos elementos se definió una estrategia que organiza controles desde la selección hasta el monitoreo y retiro.

Los lineamientos de integración se expresaron mediante etapas de repositorio, revisión,

CI/CD, construcción, distribución y operación. Finalmente, la *PoC* demostró la aplicabilidad de un subconjunto de controles en un entorno controlado. Por lo tanto, los objetivos específicos se abordaron de forma encadenada y contribuyeron al objetivo general de proponer una estrategia basada en referencias existentes.

9.8 Discusión de los resultados

En el alcance de la corrida, los resultados muestran que el ejecutor relaciona comprobaciones del registro, *npm*, integridad y reportes de *Trivy* con decisiones explícitas: los nueve escenarios riesgosos quedaron bloqueados o en revisión manual y se conservaron evidencias. La extensión de esta coordinación a una organización puede generar beneficios operativos, pero esos efectos no fueron evaluados en producción. Un escáner puede identificar una vulnerabilidad; sin una política y un responsable, el hallazgo puede no modificar el flujo. Del mismo modo, un *SBOM* facilita saber qué existe, pero requiere monitoreo y procedimientos de respuesta.

La revisión manual de S6 también resulta significativa. Una automatización que bloquee toda señal ambigua puede generar fricción y excepciones informales. La existencia de un estado intermedio permite conservar el flujo bajo control mientras un responsable analiza el contexto. Esta combinación de automatización y responsabilidad humana es coherente con el enfoque basado en riesgo [1, 44, 47].

El ensayo demuestra la precedencia de bloqueos y advertencias y conserva el motivo de cada decisión. La instalación real de dependencias de laboratorio se realizó con scripts deshabilitados; S4 verificó que el marcador de un `postinstall` no se generara. Esto demuestra prevención de la ejecución en ese ensayo, no detección general de malware, aislamiento de un runner de CI ni protección de una construcción productiva.

9.9 Limitaciones de la prueba de concepto

La corrida consulta el registro *npm* y una base de vulnerabilidades de *Trivy*, por lo que los resultados dependen de sus contenidos y versiones a la fecha observada. Los escenarios son de laboratorio y fueron seleccionados para ejercitar controles; no son una muestra aleatoria. Las instalaciones usan `-ignore-scripts`; la prueba no ejecuta scripts de ciclo de vida ni evalúa ataques con comportamiento dinámico, compromiso del propio motor, evasión de firmas, errores de configuración de una plataforma productiva o colusión de actores.

El único caso limpio no permite estimar falsos positivos representativos. No se midieron tiempos, costos ni desempeño a escala y la corrida real no se repitió para evaluar estabilidad entre actualizaciones de las fuentes. Aunque se instalan paquetes de laboratorio y se inspeccionan sus lockfiles, no se evalúa la cobertura completa de transitivas de un producto productivo. La *PoC* no analiza comportamiento malicioso dinámico, no genera un *SBOM* ni procedencia en su modo real, y no verifica firmas o identidades de publicadores. La validación externa requiere

ampliar casos e integrar controles en un entorno organizacional.

La corrida registra versiones de herramientas, fecha de observación, configuración de política y metadatos de la base de vulnerabilidades para permitir la interpretación posterior. Una nueva ejecución puede variar por la publicación de versiones, cambios en artefactos, actualizaciones de avisos o variaciones del estado de mantenimiento; por ello, la estabilidad entre corridas reales no se afirma a partir de esta única ejecución.

10. Conclusiones

La investigación partió de un problema recurrente: las dependencias aceleran el desarrollo, pero trasladan parte de la confianza hacia componentes, mantenedores, repositorios y procesos externos. El análisis realizado permite concluir que su seguridad no puede reducirse a actualizar versiones o ejecutar un escáner. Requiere un proceso gobernado que acompañe todo el ciclo de vida y produzca evidencia para decidir y responder.

10.1 Cumplimiento del objetivo general

El objetivo general se abordó mediante una estrategia para paquetes directos y transitivos que articula ciclo de vida, riesgos, controles y evidencias. *SSDF* y *SAMM* aportan estructura; *SCVS*, verificaciones sobre componentes; *SLSA*, garantías de construcción; las guías, formatos y herramientas, mecanismos de apoyo. La arquitectura y el inventario consolidado permiten identificar qué debe implementarse y cómo observarlo.

La prueba de concepto mostró que una parte representativa de la estrategia puede ejecutarse con reglas versionadas y herramientas reales. En la corrida registrada, los nueve escenarios riesgosos fueron bloqueados o derivados a revisión manual, las diez decisiones coincidieron con las esperadas y se generaron evidencias trazables. El resultado se limita a esos escenarios y no se extiende a una validación productiva.

10.2 Cumplimiento de los objetivos específicos

- OE1. Se analizaron los riesgos vinculados con dependencias directas y transitivas en repositorios, gestores de paquetes, instalación, compilación, distribución y operación, identificando amenazas de vulnerabilidad, origen, integridad, mantenimiento y licenciamiento.

- OE2. Se compararon los marcos, modelos, estándares y buenas prácticas seleccionados, lo que permitió distinguir sus alcances, limitaciones y complementariedades para la seguridad de la cadena de suministro.

- OE3. Se identificaron como principios comunes la visibilidad, la trazabilidad, la evaluación basada en riesgo, la automatización responsable, la procedencia, la asignación de responsabilidades y la mejora continua.

- OE4. Se propuso una arquitectura con repositorio, gestor o proxy, *CI/CD*, análisis de componentes, motor de políticas, *SBOM*, artefactos, firma/procedencia y monitoreo. Su relación con las funciones del ciclo y la gobernanza transversal se presenta en el capítulo 7.

- OE5. Se definieron lineamientos para traducir la estrategia en controles técnicos sobre repositorios, revisiones, *CI/CD*, generación de *SBOM*, firma, procedencia, distribución y monitoreo.

- OE6. Se evaluaron diez escenarios en una corrida real con *npm* y *Trivy*; las diez decisiones coincidieron con las esperadas, siete escenarios fueron bloqueados, dos derivados a revisión manual y el caso limpio permitido. La suite automatizada del proyecto contiene doce pruebas. El resultado verifica el comportamiento de estos controles con herramientas reales, pero no acredita eficacia general ni valida todas las capacidades propuestas.

10.3 Principales hallazgos

El primer hallazgo es que ninguna referencia cubre por sí sola todas las decisiones necesarias. Los marcos de desarrollo seguro aportan gobierno y proceso; *SCVS* profundiza componentes; *SLSA* protege construcción y procedencia; el *SBOM* aporta inventario; y las señales de proyecto ayudan a evaluar mantenimiento. La cobertura integral surge de la combinación consciente de estos aportes.

El segundo hallazgo es que visibilidad y acción deben mantenerse unidas. Conocer una vulnerabilidad o generar un *SBOM* no reduce el riesgo si no existe una regla, un responsable y un plazo. El tercero es que la automatización necesita estados diferenciados. Las condiciones objetivas pueden bloquearse, mientras las señales contextuales requieren revisión formal. El cuarto es que la evidencia debe generarse junto con el control y no reconstruirse después del incidente.

10.4 Aportes de la estrategia propuesta

El principal aporte es una secuencia de implementación que conecta etapas, roles, controles y evidencias. Esta estructura facilita que una organización comience por capacidades básicas y avance hacia procedencia verificable y monitoreo organizacional. Los niveles propuestos permiten ajustar el esfuerzo según criticidad sin convertir la estrategia en una exigencia uniforme.

Otro aporte es el modelo de decisión compuesto por permitir, bloquear y revisar. Este modelo hace visible la aceptación del riesgo y puede contribuir a reducir excepciones informales. La *PoC* aporta un ejemplo ejecutable de cómo aplicar dicho modelo a proyectos de laboratorio y reportes de herramientas reales; las pruebas del modo simulado aportan, por separado, comprobaciones deterministas del motor.

10.5 Limitaciones del trabajo

El objeto se limita a paquetes gestionados conforme al capítulo 4. La comparación utiliza ediciones y referencias seleccionadas; no es una revisión sistemática exhaustiva ni una certificación. La validación técnica utiliza proyectos de laboratorio de *npm* y una muestra *Maven*;

no representa una aplicación productiva ni valida la integración con otros gestores. La generalización hacia *APIs*, servicios cloud, bases de datos u otros entornos requiere otro diseño experimental.

Tampoco se evaluaron costos, resistencia organizacional, calidad de datos en tiempo real o desempeño a escala. Los resultados no constituyen una certificación ni garantizan ausencia de vulnerabilidades. Estas limitaciones no invalidan la propuesta, pero definen el alcance dentro del cual pueden interpretarse sus conclusiones.

10.6 Recomendaciones para su implementación

Se recomienda comenzar con un inventario confiable y propietarios definidos. Luego deben establecerse repositorios autorizados, archivos de bloqueo, análisis de vulnerabilidades y reglas simples de decisión. La automatización debe incorporarse gradualmente, midiendo excepciones y calidad de resultados. Los sistemas críticos deben priorizarse antes de extender controles a toda la organización.

Las políticas deben versionarse, probarse y comunicarse. Cada excepción necesita responsable y vencimiento. El *SBOM* debe asociarse con el artefacto real y utilizarse en monitoreo y respuesta. Para etapas avanzadas, se recomienda promover artefactos inmutables, generar procedencia desde una plataforma confiable y verificarla antes del despliegue. La capacitación y la revisión periódica de métricas deben acompañar todas las fases.

10.7 Líneas de trabajo futuro

El siguiente paso técnico es integrar el flujo con un pipeline de CI/CD y un repositorio de artefactos, generar un *SBOM* válido a partir del árbol resuelto, y verificar procedencia y firmas. El piloto debe incluir fallas de fuentes, campos ausentes, casos limpios diversos, muestras adicionales de vulnerabilidades y controles de identidad. Luego podrán evaluarse otros ecosistemas, tiempos, costos y efectos sobre los equipos.

Otras líneas incluyen integrar *VEX* con el *SBOM*, evaluar procedencia firmada en plataformas endurecidas, estudiar políticas de licencias y mantenimiento con datos reales, y automatizar el análisis de impacto durante incidentes. Finalmente, un estudio longitudinal permitiría observar madurez, comportamiento de equipos y evolución de indicadores, aportando evidencia sobre la sostenibilidad organizacional de la estrategia.

Bibliografía

[1] OWASP Foundation, "Software Component Verification Standard (SCVS), versión 1.0", 2020. [En línea]. Disponible en: https://scvs.owasp.org/. Dominios V1–V6. [Accedido: 21 sep. 2026].

[2] M. Souppaya, K. Scarfone y D. Dodson, "Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities," NIST SP 800-218, National Institute of Standards and Technology, feb. 2022, doi: 10.6028/NIST.SP.800-218. [En línea]. Disponible en: https://csrc.nist.gov/pubs/sp/800/218/final. [Accedido: 5 ago. 2026].

[3] Verizon, "2026 Data Breach Investigations Report: Executive Summary," 2026. [En línea]. Disponible en: https://www.verizon.com/business/resources/executivebriefs/2026-dbir-executive-summary.pdf. [Accedido: 5 ago. 2026].

[4] Sonatype, "2024 State of the Software Supply Chain," 2024. [En línea]. Disponible en: https://www.sonatype.com/state-of-the-software-supply-chain/2024/introduction. [Accedido: 5 ago. 2026].

[5] European Union Agency for Cybersecurity (ENISA), "Threat Landscape for Supply Chain Attacks," 29 jul. 2021. [En línea]. Disponible en: https://www.enisa.europa.eu/publications/threat-landscape-for-supply-chain-attacks. [Accedido: 5 ago. 2026].

[6] IBM Security y Ponemon Institute, "Cost of a Data Breach Report 2025," 2025. [En línea]. Disponible en: https://www.ibm.com/reports/data-breach. [Accedido: 5 ago. 2026].

[7] Cybersecurity and Infrastructure Security Agency (CISA), "Software Bill of Materials (SBOM)," 2024. [En línea]. Disponible en: https://www.cisa.gov/sbom. [Accedido: 5 ago. 2026].

[8] Cybersecurity and Infrastructure Security Agency, National Security Agency y Office of the Director of National Intelligence, "Securing the Software Supply Chain: Recommended Practices for Managing Open-Source Software and Software Bill of Materials," Enduring Security Framework, dic. 2023. [En línea]. Disponible en: https://www.cisa.gov/sites/default/files/2023-12/ESF_SECURING_THE_SOFTWARE_SUPPLY_CHAIN%20RECOMMENDED%20PRACTICES%20FOR%20MANAGING%20OPEN%20SOURCE%20SOFTWARE%20AND%20SOFTWARE%20BILL%20OF%20MATERIALS.pdf. [Accedido: 5 ago. 2026].

[9] M. Mirakhorli, D. Garcia, S. Dillon, K. Laporte, M. Morrison, H. Lu, V. Koscinski y C. Enoch, “A Landscape Study of Open Source and Proprietary Tools for Software Bill of Materials (SBOM),” arXiv:2402.11151, 2024, doi: 10.48550/arXiv.2402.11151. [En línea]. Disponible en: https://arxiv.org/abs/2402.11151. [Accedido: 5 ago. 2026].

[10] M. Zimmermann, C.-A. Staicu, C. Tenny y M. Pradel, “Small World with High Risks: A Study of Security Threats in the npm Ecosystem,” en *28th USENIX Security Symposium (USENIX Security 19)*, Santa Clara, CA, USA, 2019, pp. 995–1010. [En línea]. Disponible en: https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman. [Accedido: 5 ago. 2026].

[11] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu y X. Peng, “Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem,” en *Proc. 44th IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2022, pp. 672–684, doi: 10.1145/3510003.3510142.

[12] Microsoft Threat Intelligence, “Mitigating the Axios npm Supply Chain Compromise,” 1 abr. 2026. [En línea]. Disponible en: https://www.microsoft.com/en-us/security/blog/2026/04/01/mitigating-the-axios-npm-supply-chain-compromise/. [Accedido: 5 ago. 2026].

[13] I. Sommerville, *Software Engineering*, 10.ª ed. Boston, MA, USA: Pearson, 2016.

[14] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.

[15] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA, USA: Addison-Wesley, 1994.

[16] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[17] S. Newman, *Building Microservices*, 2.ª ed. Sebastopol, CA, USA: O’Reilly Media, 2021.

[18] C. Davis, Cloud Native Patterns. Greenwich, CT, USA: Manning Publications, 2019.

[19] M. Richards y N. Ford, *Fundamentals of Software Architecture*. Sebastopol, CA, USA: O’Reilly Media, 2020.

[20] N. Forsgren, J. Humble y G. Kim, *Accelerate: The Science of Lean Software and DevOps*. Portland, OR, USA: IT Revolution Press, 2018.

[21] J. White y D. A. Wheeler, “OpenSSF Expands Supply Chain Integrity Efforts with S2C2F,” Open Source Security Foundation, 16 nov. 2022. [En línea]. Disponible

en: https://openssf.org/blog/2022/11/16/openssf-expands-supply-chain-integrity-efforts-with-s2c2f/. [Accedido: 5 ago. 2026].

[22] Linux Foundation, "New Research from Snyk and The Linux Foundation Reveals Significant Security Concerns Resulting from Open Source Software Ubiquity," 21 jun. 2022. [En línea]. Disponible en: https://www.linuxfoundation.org/press/press-release/state-of-open-source-security. [Accedido: 5 ago. 2026].

[23] Cybersecurity and Infrastructure Security Agency, National Security Agency y Office of the Director of National Intelligence, "Securing the Software Supply Chain: Recommended Practices for Developers," Enduring Security Framework, 2022. [En línea]. Disponible en: https://www.cisa.gov/sites/default/files/2023-12/ESF_SECURING_THE_SOFTWARE_SUPPLY_CHAIN_DEVELOPERS.pdf. [Accedido: 5 ago. 2026].

[24] P. Ammann y J. Offutt, *Introduction to Software Testing*, 2.ª ed. Cambridge, Reino Unido: Cambridge University Press, 2016, doi: 10.1017/9781316771273.

[25] Open Source Security Foundation (OpenSSF), "Software Supply Chain," [En línea]. Disponible en: https://openssf.org/technical-initiatives/software-supply-chain/. [Accedido: 5 ago. 2026].

[26] R. J. Ellison, C. J. Alberts, R. C. Creel, A. J. Dorofee y C. Woody, "Software Supply Chain Risk Management: From Products to Systems of Systems," Software Engineering Institute, Carnegie Mellon University, CMU/SEI-2010-TN-026, dic. 2010, doi: 10.1184/R1/6584210.v1. [En línea]. Disponible en: https://www.sei.cmu.edu/library/software-supply-chain-risk-management-from-products-to-systems-of-systems/. [Accedido: 5 ago. 2026].

[27] National Institute of Standards and Technology, "Software Security in Supply Chains," 5 may. 2022. [En línea]. Disponible en: https://csrc.nist.gov/pubs/other/2022/05/05/software-security-in-supply-chains/final. [Accedido: 5 ago. 2026].

[28] OWASP Foundation, "OWASP Software Assurance Maturity Model (SAMM), versión 2", [En línea]. Disponible en: https://owaspsamm.org/model/. [Accedido: 21 sep. 2026].

[29] Open Source Security Foundation (OpenSSF), "Supply-chain Levels for Software Artifacts (SLSA) Specification Version 1.0," 2023. [En línea]. Disponible en: https://slsa.dev/spec/v1.0/. [Accedido: 5 ago. 2026].

[30] Center for Internet Security (CIS), "CIS Software Supply Chain Security Guide," 31 ago. 2022. [En línea]. Disponible en: https://www.cisecurity.org/insights/white-papers/cis-software-supply-chain-security-guide. [Accedido: 5 ago. 2026].

[31] Open Web Application Security Project (OWASP), “OWASP Top 10,” [En línea]. Disponible en: https://owasp.org/Top10/. [Accedido: 5 ago. 2026].

[32] Open Source Security Foundation (OpenSSF), “OpenSSF Securing Software Repositories Working Group,” [En línea]. Disponible en: https://repos.openssf.org/. [Accedido: 5 ago. 2026].

[33] CVE Program, “Common Vulnerabilities and Exposures (CVE),” [En línea]. Disponible en: https://www.cve.org/. [Accedido: 5 ago. 2026].

[34] National Institute of Standards and Technology, “CVE-2021-44228 Detail (Log4Shell),” 2021. [En línea]. Disponible en: https://nvd.nist.gov/vuln/detail/CVE-2021-44228. [Accedido: 5 ago. 2026].

[35] National Institute of Standards and Technology, “National Vulnerability Database (NVD),” [En línea]. Disponible en: https://nvd.nist.gov/. [Accedido: 5 ago. 2026].

[36] National Telecommunications and Information Administration (NTIA), “The Minimum Elements for a Software Bill of Materials (SBOM),” U.S. Department of Commerce, 2021. [En línea]. Disponible en: https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom. [Accedido: 5 ago. 2026].

[37] The White House, “Executive Order 14028: Improving the Nation’s Cybersecurity,” 12 may. 2021. [En línea]. Disponible en: https://www.federalregister.gov/documents/2021/05/17/2021-10460/improving-the-nations-cybersecurity. [Accedido: 5 ago. 2026].

[38] Linux Foundation, “SPDX Specification Version 3.0,” 2024. [En línea]. Disponible en: https://spdx.github.io/spdx-spec/v3.0/. [Accedido: 5 ago. 2026].

[39] CycloneDX Project, “CycloneDX Specification Version 1.6”, 2024. Esquema JSON: https://raw.githubusercontent.com/CycloneDX/specification/1.6/schema/bom-1.6.schema.json. [Accedido: 21 sep. 2026].

[40] National Institute of Standards and Technology, *The NIST Cybersecurity Framework (CSF) 2.0*, NIST CSWP 29, feb. 2024, doi: 10.6028/NIST.CSWP.29. [En línea]. Disponible en: https://csrc.nist.gov/pubs/cswp/29/the-nist-cybersecurity-framework-csf-20/final. [Accedido: 5 ago. 2026].

[41] Open Source Security Foundation (OpenSSF), “OpenSSF Scorecard Documentation,” 2024. [En línea]. Disponible en: https://scorecard.dev/. [Accedido: 5 ago. 2026].

[42] International Organization for Standardization, *ISO/IEC 27034-1:2011, Information Technology—Security Techniques—Application Security—Part 1: Overview and Concepts*. Ginebra, Suiza: ISO, 2011.

[43] OWASP Foundation, “OWASP DevSecOps Guideline,” 2024. [En línea]. Disponible en: https://owasp.org/www-project-devsecops-guideline/. [Accedido: 5 ago. 2026].

[44] Open Policy Agent (OPA), “Policy as Code Documentation,” 2024. [En línea]. Disponible en: https://www.openpolicyagent.org/docs/. [Accedido: 5 ago. 2026].

[45] R. Chandramouli, F. Kautz y S. Torres-Arias, *Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*, NIST SP 800-204D, National Institute of Standards and Technology, feb. 2024, doi: 10.6028/NIST.SP.800-204D. [En línea]. Disponible en: https://csrc.nist.gov/pubs/sp/800/204/d/final. [Accedido: 5 ago. 2026].

[46] G. Kim, J. Humble, P. Debois, J. Willis y N. Forsgren, *The DevOps Handbook*, 2.ª ed. Portland, OR, USA: IT Revolution Press, 2021.

[47] International Organization for Standardization, *ISO 31000:2018—Risk Management—Guidelines*. Ginebra, Suiza: ISO, 2018.

[48] FIRST, “Common Vulnerability Scoring System Version 4.0: Specification Document,” 2023. [En línea]. Disponible en: https://www.first.org/cvss/v4.0/specification-document. [Accedido: 5 ago. 2026].

[49] FIRST, “Common Vulnerability Scoring System Version 3.1: Specification Document”, 2019. [En línea]. Disponible en: https://www.first.org/cvss/v3.1/specification-document. [Accedido: 21 sep. 2026].

[50] npm, “npm audit, documentación CLI v11”. [En línea]. Disponible en: https://docs.npmjs.com/cli/v11/commands/npm-audit/. [Accedido: 21 sep. 2026].

[51] Google, “OSV-Scanner”. [En línea]. Disponible en: https://google.github.io/osv-scanner/. [Accedido: 21 sep. 2026].

[52] CycloneDX, “CycloneDX for npm”, repositorio oficial. [En línea]. Disponible en: https://github.com/CycloneDX/cyclonedx-node-npm. [Accedido: 21 sep. 2026].

[53] J. L. Slipak, “Prueba de concepto para la gestión segura de dependencias”, commit `8ae43588d38c799f67039dcd3613814d08eafdc8` de la rama `main`, repositorio del proyecto en GitHub. [En línea]. Disponible en: `https://github.com/jslipak/poc-gestion-segura-dependencias/commit/8ae43588d38c799f67039dcd3613814d08eafdc8`. [Accedido: 6 nov. 2026].

Anexo A. Prueba de concepto y ejecución registrada

Este anexo identifica la versión del código y documenta la corrida real disponible, sus resultados y las evidencias conservadas. El manifiesto de la ejecución registra los tamaños y hashes de los archivos generados.

La validación principal utiliza proyectos de laboratorio y herramientas reales, consulta el registro *npm* y la base de vulnerabilidades de *Trivy*, e instala paquetes con scripts deshabilitados. Los resultados se asocian a la fecha, versiones y fuentes registradas en la corrida; no constituyen una certificación de seguridad ni un despliegue productivo.

A.1 Detalle de implementación, escenarios, evidencias y limitaciones por control

La tabla siguiente complementa el inventario consolidado de la sección 7.5 (Tabla 7). Para cada control se indica su implementación, los escenarios que lo ejercen, las evidencias y las limitaciones. Las rutas de evidencia son relativas a `poc_gestion_dependencias/`. Los estados son: **E** = ejecutado con herramientas o comprobaciones reales en el ensayo de laboratorio; **P** = representación parcial, sin integración ni garantía operacional; **N** = propuesto sin validación en la *PoC*. E no significa eficacia general probada frente a ataques reales.

Tabla A.1: Detalle de implementación, escenarios, evidencias y limitaciones por control (Anexo A)

ID	Implementación y evidencia	Escenarios PoC	Evidencias generadas	PoC
C01	Política versionada en config/policy.json; sin aprobación organizacional ni matriz de roles aplicada.	S0–S9	results.json (versión y parámetros)	P
C02	Solicitud de incorporación con finalidad, propietario y alternativas.	–	–	N
C03	Configuración efectiva y URLs del lockfile comparadas con registros aprobados; S2 se bloquea antes de descargar.	S0, S2, S9	registry-inspection.json	E
C04	`npm ci -ignore-scripts`; comprueba instalación fijada y rechaza lockfile inconsistente.	S0, S3	npm-ci.json, package-lock.json	E

Continúa en la siguiente página

Tabla A.1 – continuación

ID	Implementación y evidencia	Escenarios PoC	Evidencias generadas	PoC
C05	Proyectos, manifiestos, lockfiles e inventario reportado por el escáner; no se genera SBOM en el modo real.	S0–S9	projects/, results.json, reportes Trivy	P
C06	*Trivy* analiza vulnerabilidades con su base actual; la política bloquea CRITICAL/CVSS $\geq$ 9 y deriva HIGH/CVSS $\geq$ 7 a revisión.	S1, S8, S9	trivy-report.json, trivy-execution.json	E
C07	Inspección estática de scripts de ciclo de vida; instalación con scripts deshabilitados y marcador ausente.	S4	script-inspection.json, npm-ci.json	E
C08	Integridad *SHA-512 SRI* publicada por *npm* frente a tarball original y alterado.	S5, S9	trusted-reference.json, archivos .tgz	E
C09	Antigüedad del último release consultada al registro; señal de revisión, no prueba de abandono.	S6	maintenance-source.json	E
C10	*Trivy* analiza licencias y compara contra la lista permitida.	S7	trivy-report.json, trivy-execution.json	E
C11	El ejecutor combina hallazgos y aplica precedencia: bloquear, revisión manual, permitir.	S0–S9	results.json, summary.csv	E
C12	El modo simulado genera un SBOM demostrativo; no se produce ni valida SBOM en la corrida real.	S0–S9 (simulado)	sbom.cdx.json (simulado)	P
C13	El modo simulado genera procedencia sin firma ni construcción real; ausente en modo real.	S0, S6, S8 (simulado)	provenance.intoto.json	P

Continúa en la siguiente página

Tabla A.1 – continuación

ID	Implementación y evidencia	Escenarios PoC	Evidencias generadas	PoC
C14	Servicio de firma, identidad autorizada y verificador. Sin implementación en la *PoC*.	–	–	N
C15	*MFA*, ramas protegidas, revisión, mínimo privilegio y registros de acceso.	–	–	N
C16	Runners aislados, herramientas fijadas y secretos limitados. No evaluado localmente.	–	–	N
C17	Repositorio de artefactos por hash y verificación previa al despliegue.	–	–	N
C18	La salida permite revisión manual, pero no implementa una aprobación con propietario y vencimiento.	S6, S8	results.json (decisión)	P
C19	Inventario desplegado, alertas, tareas de remediación y registro de incidente.	–	–	N
C20	Gestor y solicitudes de cambio; prueba de eliminación directa y transitiva.	–	–	N
C21	Formación por rol, revisión de métricas y ajuste de políticas.	–	–	N
C22	Manifiesto SHA-256 de evidencias guardadas localmente; sin repositorio inmutable ni custodia protegida.	S0–S9	evidence/evidence-manifest.json	P

Notas sobre la Tabla A.1:

- Los controles con estado **E** (C03, C04, C06–C11) se ejecutaron en los escenarios reales indicados, utilizando *npm*, consultas al registro y/o reportes de *Trivy*.
- Los controles con estado **P** (C01, C05, C12, C13, C18, C22) tienen representación parcial. El SBOM y la procedencia pertenecen al modo simulado; no son artefactos de la

corrida real. No se implementan aprobación organizacional, inventario completo de transitivas, firma, tramitación de excepciones ni custodia protegida.

- Los controles con estado **N** (C02, C14–C17, C19–C21) se proponen como capacidades organizacionales y técnicas cuya validación requiere un piloto posterior con infraestructura real (proxy, CI/CD, *SCA* externo, repositorio de artefactos, servicio de firma, monitoreo productivo).
- Las evidencias de la corrida se encuentran bajo `poc_gestion_dependencias/output/real-run/`; el manifiesto global enumera los archivos, tamaños y SHA-256. La salida del modo simulado reside bajo `poc_gestion_dependencias/output/` y se mantiene separada.
- La correspondencia con las referencias (fundamento) se mantiene en la Tabla 7 del capítulo 7.

A.2 Identificación de la versión y disponibilidad

El código utilizado corresponde al proyecto `poc_gestion_dependencias`. El archivo `package.json` identifica la versión 2.0.0 y `config/policy.json` contiene la política 2.0.0. El comando predeterminado ejecuta el ensayo real. El modo simulado se conserva como mecanismo separado para pruebas deterministas del motor y no forma parte de los resultados informados en este anexo. El repositorio del proyecto está en GitHub [53].

El código publicado y cotejado en la rama `main` corresponde al commit `8ae43588d38c799f67039dcd3613814d08eafdc8`, disponible mediante el enlace permanente de la referencia [53]. El proyecto declara versión 2.0.0 en `package.json` y la política también es 2.0.0; estos valores identifican el software y la política, no un commit. El tag Git `v2.0.0` apunta al commit anterior `8259e02807045c4daab77f779ace5b2d8e99acfc`, que no incluye `src/real-runner.js`; por eso se fija el commit de `main` para identificar la versión utilizada. El PDF de la tesis no forma parte del repositorio.

La salida de cada corrida depende de los datos obtenidos del registro y de la base de vulnerabilidades. Por ello, no se presenta un hash fijo de un paquete ZIP como identificador de estos resultados. El manifiesto `poc_gestion_dependencias/output/real-run/evidence/evidence-manifest.json` registra tamaños y SHA-256 de los proyectos, lockfiles, reportes y resultados de la corrida concreta.

A.2.1 Ejecución registrada

La corrida documentada se observó el 6 de noviembre de 2026 a las 23:40:19 UTC. Se utilizaron *Node.js* v24.22.1, *npm* 11.11.0 y *Trivy* 0.76.2. La base de vulnerabilidades de *Trivy* era la versión 2, actualizada el 6 de noviembre de 2026 a las 18:59:52 UTC y descargada durante la

corrida. La política aplicada fue `POL-DEPENDENCIAS-001`, versión 2.0.0, con bloqueo a partir de CVSS 9, revisión manual a partir de CVSS 7 y antigüedad máxima de mantenimiento de 730 días. El resultado conserva el SHA-256 calculado por el ejecutor sobre la serialización JSON de la política: `244cdefeb16a81fe28727caaf81b57a43b3d82a5c0053c77883060dbf2193bd9`.

La ejecución consultó `registry.npmjs.org` y la base de vulnerabilidades de *Trivy*. Los scripts de ciclo de vida se deshabilitaron mediante `-ignore-scripts`. Estos datos describen la corrida preservada; una ejecución posterior puede producir versiones o hallazgos diferentes.

A.3 Requisitos y procedimiento de ejecución

- **Requisito 1.** *Node.js* 18 o superior y *npm*.
- **Requisito 2.** *Trivy*; la corrida documentada utilizó la versión 0.76.2.
- **Requisito 3.** Conexión a internet para consultar el registro *npm* y actualizar la base de vulnerabilidades de *Trivy*.
- Desde la carpeta `poc_gestion_dependencias` se ejecuta el modo real y la suite automatizada:

```
npm run poc
npm test
```

El comando `npm run poc` genera los proyectos de laboratorio, analiza los escenarios y guarda los resultados bajo `output/real-run/`. `npm test` ejecuta pruebas automatizadas del motor y de sus funciones auxiliares. `npm run poc:simulated` selecciona el modo simulado, que conserva datos separados y no debe confundirse con el ensayo real.

A.4 Demostración y resultados observados

La Tabla A.2 detalla las decisiones esperadas y obtenidas por escenario; la Tabla A.3 resume los indicadores generales de esta corrida.

La concordancia de las decisiones corresponde a los escenarios definidos y a esta corrida; no estima tasas de detección o error para una población de paquetes. S0 es el único caso limpio. La corrida no incluye una línea de base productiva ni una repetición del ensayo real.

A.5 Inventario de evidencias

El directorio `poc_gestion_dependencias/output/real-run/` conserva `results.json`, `summary.csv` y los proyectos generados con sus manifiestos y lockfiles. En `evidence/` se guardan los reportes JSON originales y comandos de *Trivy*; además, según el escenario, la inspección del registro, la salida de `npm ci`, la inspección estática de scripts, la fuente de antigüedad y la referencia de integridad. El escenario S5 conserva

Tabla A.2: Resultados de la ejecución de los escenarios reales de laboratorio

Escenario	Control observado	Esperado	Obtenido
S0	Caso limpio; paquete `chalk@6.0.1`, lockfile, licencia, scripts y escaneo	Permitir	Permitir
S1	`log4j-core:2.14.1`; vulnerabilidades críticas informadas por *Trivy*	Bloquear	Bloquear
S2	Registro efectivo y URLs de resolución no autorizados	Bloquear	Bloquear
S3	`npm ci` rechaza manifiesto y lockfile inconsistentes	Bloquear	Bloquear
S4	`postinstall` detectado; el marcador confirma que no se ejecutó	Bloquear	Bloquear
S5	Tarball original de `lodash` frente a copia alterada e integridad publicada	Bloquear la copia alterada	Bloquear
S6	Último release de `request` con antigüedad de 2460 días	Revisión manual	Revisión manual
S7	Licencia `GPL-3.0-only` identificada por *Trivy*	Bloquear	Bloquear
S8	Vulnerabilidades altas en `path-to-regexp@0.1.7`	Revisión manual	Revisión manual
S9	Vulnerabilidades, registro no autorizado e integridad alterada	Bloquear	Bloquear

Tabla A.3: Síntesis de la corrida registrada

Indicador	Resultado
Decisiones coincidentes con lo esperado	10 de 10 escenarios
Escenarios riesgosos con decisión no permisiva	9 de 9: siete bloqueados y dos derivados a revisión manual
Caso limpio	S0 permitido sin hallazgos
Reportes de vulnerabilidades y licencias	Reportes JSON originales de *Trivy* para los escenarios analizados
Pruebas automatizadas del proyecto	12 aprobadas en la suite ejecutada
Repetibilidad del ensayo real	No medida; las fuentes externas pueden cambiar

los tarballs original y alterado. El manifiesto global `evidence/evidence-manifest.json` registra tamaños y SHA-256 de los archivos de entrada y salida incluidos en la corrida.

Los hashes del manifiesto permiten detectar cambios respecto de los bytes registrados, pero no prueban por sí solos la identidad del publicador ni proporcionan una firma o custodia inmutable. El modo real no genera un *SBOM* ni una declaración de procedencia; las versiones demostrativas de esos artefactos pertenecen al modo simulado.

A.5.1 Inventario verificable de la corrida

La Tabla A.4 conserva las 56 entradas del manifiesto de evidencias de esta ejecución, con ruta, tamaño y SHA-256. El propio manifiesto no se incluye en su lista para evitar una referencia circular.

Tabla A.4: Inventario verificable de evidencias de la corrida real

Archivo	Tamaño	*SHA-256*
`output/real-run/artifacts/lodash-4.17.21-alterado.tgz`	318,961 bytes	`0206b0a0e3713bad835121049e8f6aff943bcd5ee0289c3a778dce78243ced82`
`output/real-run/artifacts/lodash-4.17.21.tgz`	318,961 bytes	`6a087ac9e5702a0c9d60fbcd48696012646ec8df1491dea472b150e79fcaf804`
`output/real-run/evidence/S0/baseline-controls.json`	345 bytes	`fb2a225b9d1fb143316b1992a73c5ecf7e6c7725ab414d9bc782743563ec0c96`
`output/real-run/evidence/S0/clean-package-source.json`	344 bytes	`284b80654d5c4b03e4542d3e308be659d1b16a6073983033113fa26425b046d7`
`output/real-run/evidence/S0/npm-ci.json`	248 bytes	`bedbcf26572a93183ea7bdcf6f929569a3fb7dd3f3eeef0308560512931e53d3`
`output/real-run/evidence/S0/registry-inspection.json`	310 bytes	`1f25a557a4c8de29526a8d3352cc43060d7501678cc02b8542aaddee421d17ac`
`output/real-run/evidence/S0/trivy-execution.json`	330 bytes	`e63026be3cb201c4b4cc54a23a52a2b16ef712c5f77be872a05ab29166c9f9a1`
`output/real-run/evidence/S0/trivy-report.json`	1,229 bytes	`b0bee60266801bb97f97507bfd0c35fae9e8ed095f4cb5a0d02f4a9180b8e7a0`
`output/real-run/evidence/S1/trivy-execution.json`	330 bytes	`7d8787b4839dbfd543772f603494772e32c4b2b4ff124d018581cca9f8f02bd1`
`output/real-run/evidence/S1/trivy-report.json`	48,342 bytes	`9cf4ddcb1bfe3881c90192453e28d9f332096266d658ab5b031f4cf87236c7a4`
`output/real-run/evidence/S2/registry-inspection.json`	329 bytes	`e85fd0d1312151c4ad69ec6de196df5c38015150a0b83dc54f8dfe4a252f1455`
`output/real-run/evidence/S2/trivy-execution.json`	330 bytes	`3a91f4c4006f777ba666dec45408326ca3c74a98269743eda4054712f5ce0436`
`output/real-run/evidence/S2/trivy-report.json`	1,229 bytes	`e283168f22fbb7fc3e661dad7351469eede12ce752774bcb6ffd8c97f75debab`
`output/real-run/evidence/S3/npm-ci.json`	3,543 bytes	`fb11711b3cc71ba3581015af4b6b1a0b025841b35c6b59b39b5a222d706b896b`
`output/real-run/evidence/S3/registry-inspection.json`	310 bytes	`1f25a557a4c8de29526a8d3352cc43060d7501678cc02b8542aaddee421d17ac`
`output/real-run/evidence/S3/trivy-execution.json`	330 bytes	`134d8b195f02b2b1c5554db702c7104acc1725fbe809c9b2a23673d4119b4b6f`

Continúa en la siguiente página

Tabla A.4: Inventario verificable de evidencias (continuación)

Archivo	Tamaño	*SHA-256*
output/real-run/evidence/S3/trivy-report.json	1,229 bytes	94c1dd15f0ab434cd921fae7b3c6e93f565ea97833db29fcebd4f8eb72a6802f
output/real-run/evidence/S4/npm-ci.json	248 bytes	48596e7ec63e0ae2e1f6031045eaeb858ae72e9c05eca971eb4f120b417ea889
output/real-run/evidence/S4/registry-inspection.json	274 bytes	9b4994de6173f4bef39e98b23e14b53f0c5105c9e07b07fd1af829484c2f4f66
output/real-run/evidence/S4/script-inspection.json	278 bytes	dd044cd72fc890428723c8d72b5e5f76096b858cd6be7a26bed47a337aeca2d0
output/real-run/evidence/S4/trivy-execution.json	330 bytes	12e655cc2ddaa022e67802112ec0f3f7c5ceb3206de3edda13e5aca06324615d
output/real-run/evidence/S4/trivy-report.json	1,301 bytes	998535afbecbc6006ffc4631eadafa84794f68b58256a1abc656ccb42493ceb7
output/real-run/evidence/S5/trusted-reference.json	698 bytes	5ad2e3014f7c582411a057eb5bab8c2293ffebb1f63432dc66850f3ea7e6c1fe
output/real-run/evidence/S6/maintenance-source.json	311 bytes	99c20616ebaaf97ba8f32a0656ad7c20f04f8d62b567dc190f72c5c256b292f0
output/real-run/evidence/S7/npm-ci.json	248 bytes	f0894a1f952420ce7c50fed6dbeee92e2ede53b61b8838fefbfa27f08bcccc5c
output/real-run/evidence/S7/registry-inspection.json	394 bytes	aac6ec9ece5b7b6b48d27a46f7ee6185a4e4cadb394e78a7ff3e74a4b94f4f6c
output/real-run/evidence/S7/trivy-execution.json	330 bytes	c4b6680bcb052d507e417b87fea1a3e168e53350f563962c6f93bac467dfc958
output/real-run/evidence/S7/trivy-report.json	1,361 bytes	ad6c56e822843832c973781a1b9898f418e09ccce2ce86b7ce350a1af3e65339
output/real-run/evidence/S8/npm-ci.json	248 bytes	30904537de8b4b602fe96d1aaddb34aa63d81f3a591b8804b7e61867f88d1691
output/real-run/evidence/S8/registry-inspection.json	346 bytes	73ef89bcc3c2d47e22b8b0e3cd53dc40ac5c4a23d3ad40972b869107ff5b9b34
output/real-run/evidence/S8/trivy-execution.json	330 bytes	13438802809c25c0c5349d0c3cb0c64dd323e8370a72a105adb95be39b10476d
output/real-run/evidence/S8/trivy-report.json	11,363 bytes	0479f163350f00562fc092287dbda3a676f1df00c70298fab53871f0c83a169c
output/real-run/evidence/S9/npm-ci.json	248 bytes	f917dfbc5ec0a8d9e299c6c85ccce5790f984e3a775eeffb68ee2ca8de294425
output/real-run/evidence/S9/registry-inspection.json	337 bytes	35cb444fff9634eea85dfc982dc3ee64050cd716782655961c762907f9ffbae7
output/real-run/evidence/S9/trivy-execution.json	330 bytes	a1f6c6f7f97725f12f0cd1932a4a33f1e6bf99e52dc508579ecaf65680498664
output/real-run/evidence/S9/trivy-report.json	19,584 bytes	7ef3cf4190156f284fdfeafcad2c8445016de45cd06707fa131bf97692daaf70
output/real-run/projects/S0/package-lock.json	652 bytes	41e7e424a1ae65b13699fe78da70ede659b008451da8f4f5709e7f900b24919d
output/real-run/projects/S0/package.json	114 bytes	42e27f41908b232ca62501e0cd79bf3288187ff74042ca4115ac09141deb6945
output/real-run/projects/S1/pom.xml	405 bytes	5974e2de079f908c6370984fe7e1ba4eacb925111165cc96e45e1b56abea4f48
output/real-run/projects/S2/.npmrc	43 bytes	1800cc9f226c8b773182f47edf31622eb03d24d795bd68b6d403962f2a21d7fa

Continúa en la siguiente página

Tabla A.4: Inventario verificable de evidencias (continuación)

Archivo	Tamaño	*SHA-256*
output/real-run/projects/S2/package-lock.json	658 bytes	5aa53d2a596e5f8802c52911b5de5553af65f65ac1b1ef7bde755892e40ef1b1
output/real-run/projects/S2/package.json	114 bytes	bef0b8488b2bf78be1bd3a12053c5492b4c5677d8601fca215b85e31488e8b94
output/real-run/projects/S3/package-lock.json	652 bytes	5c3e8004aad01ef29f0a551e21c7adf0ddcb39361234f3f3dd2985a994624f58
output/real-run/projects/S3/package.json	114 bytes	8a3d184da45bf4fe0282ed370ae7a8a23ffdef5a578566ec812b4c8f8d505f6f
output/real-run/projects/S4/fixtures/lab-postinstall-fixture/package.json	219 bytes	d749369d356d91a278e66baaf5b0b1bb67c41723245258df3817a8d52f2f770d
output/real-run/projects/S4/package-lock.json	549 bytes	b05bf81e363502f4a96050ba94ddec8f15bb9cd643a252a3fe04aa8746f9ef0a
output/real-run/projects/S4/package.json	166 bytes	e7860c798755a14a85b17de99029f6f07f5b6416e083936bb936e3f1e3d2cf93
output/real-run/projects/S7/package-lock.json	620 bytes	c8a90170579f925900400b03eec5cebfdec77c4f70aa8fad8068047e8e47664a
output/real-run/projects/S7/package.json	141 bytes	e2f686ce94dce91d43007927e3a8d512db6376318d92e3338238baa059d5277f
output/real-run/projects/S8/package-lock.json	551 bytes	821a3070cb4bce318804ef091be1da5c0f5a1eab15f11097a0aabd9b1e3bfd19
output/real-run/projects/S8/package.json	123 bytes	8fa0d54efb6f729faa9928b81f0c4a0ca4d37fc6aec5ca58a11a873b02c5805f
output/real-run/projects/S9/.npmrc	43 bytes	1800cc9f226c8b773182f47edf31622eb03d24d795bd68b6d403962f2a21d7fa
output/real-run/projects/S9/package-lock.json	531 bytes	5536789fddba143dc13eddf275e38c3da0c00ffc20609a5d13d1228dd9c8b8a4
output/real-run/projects/S9/package.json	117 bytes	b2309b841bdfec6a977e823c847d2705574fcf2b5f02f6e0706f92538531bd6c
output/real-run/results.json	19,220 bytes	9bc884e9df2c1489393cd8ecead66a64503d6af8b13a1584fa730109392d57d5
output/real-run/summary.csv	2,404 bytes	f6b002c6566c89399cff346a11db8eb22fb5aa657227759f4a2168fa8cd4598f